

Trade Workbench

Automated stock and options trading system

Detailed architecture, trading lifecycle, data contracts, risk controls, user interfaces, implementation tasks and validation gates.

Prepared for your trading-system project

Research checked October 6, 2026 • Revision 1

Core design

The OpenAI assistant researches and proposes. Typed services validate, authorize, reserve risk, execute and reconcile.

Start with a trustworthy account

Build portfolio truth, risk analysis, paper/replay and fault recovery before bounded live automation.

Companion design pack

17-view interactive screen prototype; 116 implementation tasks; 56 high-impact test cases; starter SQL, strict proposal schema and strategy definition.

Status

No broker or model connection has been configured. No orders were submitted. Prototype data is synthetic.

Reading guide

System blueprint

Goals, platform research, broker capabilities, architecture, data, AI, strategies, execution, risk, portfolio, communication, operations, validation, roadmap and costs.

Screen specification

Screen purposes, layouts, fields, state handling, interactions, responsiveness and acceptance criteria.

Application contracts

Internal API routes, IDs, command states, errors, event envelopes and writer pseudocode.

Test matrix

High-impact faults and stage-specific release gates.

Source register

Official broker, OpenAI, platform, data and regulatory documentation.

Implementation handoff

Repository boundaries, initial vertical slice and implementation rules.

Use the PDF bookmarks to navigate. The complete ZIP includes editable Markdown, CSV tasks, standalone screen prototype and starter contracts. Proposed policies, timings and budgets are assumptions that must be calibrated; no strategy edge is established by this design.

Automated Stock and Options Trading Workbench

Implementation blueprint, revision 1 — researched October 6, 2026

Working product name: Trade Workbench.

This is a system design and implementation specification. No brokerage account has been connected, no market subscription purchased, and no orders submitted. All prototype balances, quotes, positions, dates, returns, and event names are illustrative. Proposed policies are engineering starting points that need account-specific calibration; they do not establish a profitable strategy.

1. Product goal and starting assumptions

Build a personal trading operating system that combines a trustworthy view of the brokerage account, market research, stock and options analytics, systematic strategies, bounded automation, and an OpenAI assistant. Its central output is a traceable decision: what the system observed, what it proposed, which rules passed or failed, what was submitted, what actually filled, and what changed in the account.

Initial scope: one U.S. individual trading their own tastytrade accounts, U.S. stocks and ETFs, equity and supported index options, USD reporting, swing trading and selected intraday strategies. Support cash and margin accounts through explicit account capabilities. A multi-user product, futures, futures options, crypto, international tax reporting, and institutional routing are expansion paths. Retail broker connectivity and model latency make this unsuitable as a high-frequency trading engine.

Use \$10,000 only in worked examples. Do not infer a funded balance, options approval, desired leverage, or authorized live budget from previous conversations. Discover all of those from the account and the user's eventual configuration.

The best system is not the one that trades most often. Optimize reliability, research integrity, net performance after costs, usable decisions, and bounded operational failure. Allow the strategy to choose no trade. An idle strategy can be working correctly.

1.1 Non-negotiable engineering properties

1. Brokerage credentials stay in a server-side vault. The browser, model prompts, research notebooks, logs, and third-party notification payloads never receive them.
2. All account-changing commands pass through one controlled broker adapter. Manual UI orders and automated orders use the same validation path.
3. The AI may propose and explain. Deterministic services compute quantities, valuation, margin effects, risk decisions, authorization, and submission.
4. Read-only, broker sandbox, realistic paper, shadow, supervised live, and bounded live are visibly different environments. A portfolio has an environment identity, not just a colored label.
5. An ambiguous order submission becomes an unresolved incident. No automatic blind retry of a money-moving request.
6. Unknown data remains unknown. Missing earnings, invalid Greeks, stale quotes, and incomplete contract metadata cannot be silently converted into safe values.
7. Approved automation is bound to an immutable strategy version, an account, risk policy, product universe, capital limit, and validity period.

- 8. Stops and loss limits are operating controls, not a guarantee of a maximum realized loss. Gaps, assignment, outages, liquidity, and fees can exceed intended limits.
- 9. Every trade has a provenance chain and a management plan. External/manual positions remain visible even if the system did not create them.
- 10. Existing positions keep appropriate deterministic monitoring when the AI provider is unavailable. New AI-dependent entries can be suspended independently.

1.2 Measurable success

Measure order uncertainty, account reconciliation mismatches, duplicated executions, data age at decision, rejected submissions, slippage versus arrival price, fees, buying-power headroom, scenario losses, strategy drift, and alert acknowledgment. Measure user workflow completion: inspect a position, explain a trade, preview a portfolio change, pause a strategy, and recover after a disconnect. Trading performance is assessed separately with out-of-sample evidence and uncertainty intervals.

2. Existing products to study and what to adopt

These are researched design patterns, not copied proprietary UI or source code. Sources are listed in 10_Source_Register.md.

Product	Useful pattern	Our implementation decision
TradingView	Synchronized charts, watchlists, screeners, alerts, command search, replay	One research workspace with linked symbol selection; chart data comes from licensed feeds; reuse a chart library only under its license
OptionStrat	Multi-leg strategy construction, payoff exploration, candidate optimization	Build a visual trade composer with synchronized legs, expiry payoff, before-expiry repricing, and portfolio impact; label all model assumptions
Option Alpha	Scanner/monitor workflows, reusable automation, logs, backtest-to-bot continuity	Visual rule editor compiles to one versioned strategy definition; the same decision code runs in replay, paper, shadow, and live
Interactive Brokers Risk Navigator	Portfolio drill-down and hypothetical changes in price, time, volatility	Dedicated Risk Lab with before/after portfolio comparison and inspectable assumptions
QuantConnect / LEAN	Event-driven research and live brokerage integration	Evaluate LEAN against our required complex-order and risk workflows before choosing the research engine; do not reinvent a general backtester without a documented gap
TradeZella	Journaling, setup analysis, replay, performance breakdowns	Generate a journal from decisions and executions automatically; add notes without altering the original evidence
tastytrade	Options-centric account, chain, trade, and buying-power workflows	Keep broker-native concepts recognizable and provide a clear handoff to its application for unsupported actions

The integration opportunity is substantial: QuantConnect documents tastytrade live support, and tastytrade now documents an options backtesting service plus a self-hosted MCP server. Evaluate all three. Use the MCP server as an optional read-only assistant integration initially; our production execution authority still lives in our application. Verify feature parity, permissions, environments, licensing, and supported order types with contract tests. [S01-S14]

3. Verified broker capabilities and engineering consequences

3.1 Authentication and account access

Current tastytrade documentation requires OAuth, with 15-minute access tokens and required User-Agent headers. Production and certification environments use separate credentials. A personal OAuth application is restricted to its owner's account; a product accepting other users needs tastytrade's third-party verification. [S01]

Implement a TokenManager with encrypted refresh-token/client-secret storage, expiry tracking, one refresh in flight per credential set, redacted logging, and an environment-bound client. Refresh using returned expiry metadata with a configurable safety margin. Do not hard-code a promise that refresh grants never change behavior; validate the current token response contract. Never log JWTs. If authorization is revoked, mark the connection unhealthy, suspend new entries, and alert the user with a broker-app fallback.

For personal setup, guide the user to create an OAuth application and personal grant, then enter secrets into the protected configuration flow. For eventual multi-user onboarding, use the verified authorization-code flow with state validation, exact redirect allowlisting, secure server-side exchange, and account discovery. Do not collect the brokerage password in our UI. Request read permission for monitoring; trade permission is enabled only for execution workers selected by the user.

3.2 Integration contract inventory

All paths below are broker paths, not the application's proposed API. Fetch the current OpenAPI schemas and exercise a certification contract test before implementing any mutation. Some indexed reference pages and explanatory guides disagree; the contract gate in section 3.5 exists to resolve this.

Purpose	Broker operation	Adapter behavior
Access token	POST /oauth/token	Token refresh belongs to TokenManager
Accounts	GET /customers/{customer_id}/accounts	Resolve supported current-user alias/identity in discovery
Balances	GET /accounts/{account_number}/balances	Keep currencies, effect fields, broker timestamp
Positions	GET /accounts/{account_number}/positions	Normalize quantities and directions without losing raw payload
Restrictions	GET /accounts/{account_number}/trading-status	Account capability gate and rule-regime evidence
Quote stream token	GET /api-quote-tokens	Use returned stream URL and token expiry
Quote snapshot	GET /market-data/by-type	Batch instruments, record age and source
Option chain	GET /option-chains/{symbol}/nested	Instruments and streamer identifiers from broker metadata
Market sessions	GET /market-time/equities/sessions/current and session/holiday reference	Early closes and closures affect schedules
Entry preflight	POST /accounts/{account_number}/orders/dry-run	Persist costs, buying-power impact, warnings and schema version
Submit	POST /accounts/{account_number}/orders	Single writer, immutable payload, correlation identifier

Purpose	Broker operation	Adapter behavior
Inspect	GET /accounts/{account_number}/orders/{id}	Order state and fill recovery
Reconcile orders	GET /accounts/{account_number}/orders and /orders/live	Read both recent history and active state with pagination
Cancel	DELETE /accounts/{account_number}/orders/{id}	Cancel requested is distinct from cancellation confirmed
Replace	PUT /accounts/{account_number}/orders/{id}	Preserve original leg semantics; confirm current edit restrictions
Edit preflight	POST /accounts/{account_number}/orders/{id}/dry-run	Validate allowed edits against the current contract
Brackets/related orders	/accounts/{account_number}/complex-orders and corresponding dry-run	Separate capability for OTO/OCO/OTOCO/PAIRS; instrument-specific tests
Broker margin view	GET /margin/accounts/{account_number}/requirements	Buying power is not the same as economic loss
Hypothetical margin	POST /margin/accounts/{account_number}/dry-run	Compare broker margin effect with our scenarios
Ledger	GET /accounts/{account_number}/transactions	Fees, fills, cash movements, corporate actions and reconciliation

Paths are grounded in the broker's guides and reference groups [S02–S09]. The table is an adapter inventory, not a claim that all capabilities are available to every account.

3.3 Streaming

Use two long-lived connections with independent health: DXLink for quotes/Greeks and the account streamer for order/account updates. A working quote stream does not prove the account stream is healthy. The market connection obtains its URL from the quote-token response and uses the documented handshake. Instrument definitions provide stream identifiers; do not construct them from display tickers. A single backend subscription manager fans data out to strategies and browser sessions. [S02, S03]

Current documentation publishes market-stream limits, while REST thresholds are not a reliable universal published per-endpoint quota. Keep provider limits in a versioned capability registry, cache slow data, batch subscriptions, and implement exponential backoff with jitter. Reserve capacity for open positions and execution candidates before research watchlists. Never open one broker stream per UI tab. [S04]

Subscription priorities: open positions and closing orders first; imminent entry candidates second; visible chain and selected chart third; active strategy universes fourth; idle watchlists last. Count subscriptions by event type, not just instruments. Deduplicate reference-counted subscriptions and apply a grace period before unsubscribe to prevent scrolling-induced churn.

3.4 Four different testing contexts

Context	Purpose	What it proves	What it does not prove
Broker certification sandbox	Validate auth, request shapes, statuses, account synchronization	Adapter and lifecycle compatibility	Tradable prices, realistic fills, profitability
Historical backtest	Execute a frozen strategy on historical inputs	Historical behavior under declared assumptions	Future returns or model-free execution realism
Realistic paper trading	Run live signals with our simulated execution	Live timing, data validity, scheduling, probable costs	Actual queue position or exchange fills
Shadow trading	Generate intents alongside a real portfolio without routing	Portfolio-aware decisions and divergence	Execution performance without live fills

tastytrade documents synthetic certification fills, unavailable sandbox market-data routes, and daily state resets. Use separate production read-only data credentials when required for realistic simulations. Sandbox fill rules belong only to sandbox fixture tests; never inject them into performance research. [S05]

The documented tastytrade backtester runs at <https://backtester.vast.tastyworks.com>, with available-date discovery and backtest, log, and single-trade simulation operations. Test coverage, strategy expressions, cost assumptions, corporate-action handling and live parity before adopting it. Archive its original outputs alongside our normalized results. Our own quote-based replay remains necessary for unsupported strategy logic and independent validation. [S06]

3.5 Capability discovery and unresolved contracts

Create `broker_capabilities` records with account, environment, capability, schema hash, evidence, verification date, status, and expiry. Enabled status requires a successful test, not a documentation link alone.

Resolve at implementation kickoff: current-user identity/account alias; option exercise and do-not-exercise support; short stock availability; fractional equity eligibility; complex brackets for each instrument class; option contract units and rounding; streamer ordering/replay guarantees; margin evaluation latency; order search completeness; third-party scope approval; options backtester coverage/entitlement; historical quote retention; and exact cancel/replace behavior.

There is a concrete documentation conflict: some reference banners tell clients to send `ext-client-order-id`, while the idempotency guide says that field is system-populated and identifies `external-identifier` as the request correlation field. Use the current request schema and sandbox tests to settle it. Disable live capability if unresolved. Neither field should be assumed to provide broker-side deduplication. [S07, S08]

Day-trading rules also need a live configuration source. FINRA describes a transition from PDT rules to new intraday-margin requirements, with brokerage implementation varying during the transition period. Do not hard-code a universal \$25,000 gate or assume it has disappeared for this specific account. Store the broker-confirmed regime and query trading status. [S24]

4. Product map and completeness boundaries

Organize the application around decisions and work, with optional advanced tools revealed inside each area.

Area	Baseline capabilities	Advanced capabilities
Overview	Account value, deployed risk, pending actions, active strategies, system state	Custom workspace layouts, household aggregation
Markets	Watchlists, charts, basic scan filters, catalysts	Linked charts, relative strength, breadth, factor analysis, custom indicators
Research	Filings, licensed news, evidence-based briefs, scenarios	Thesis tracking, research experiments, alternative datasets
Options	Chain, contract metadata, Greeks, IV and liquidity views	Skew/term structure, vol surface diagnostics, flow with limitations
Trade builder	Stock and option tickets, payoff, preflight, exit plan	Strategy optimizer, roll comparisons, custom scenario distributions
Portfolio	Positions, strategy groups, lots, orders, cash and P/L	Exposure trees, hedge comparison, return attribution
Risk Lab	Limits, concentration, stress scenarios, buying power	Historical scenarios, CVaR, covariance ensembles, sensitivity surfaces
Automation	Strategy definitions, scanner/monitor rules, budgets, mode controls	Visual recipes, parameter sweeps, regime switching, portfolio allocation
Research Lab	Backtests, replay, paper/live comparison, experiment registry	Walk-forward studies, robustness maps, experiment screening
Orders	Blotter, reconciliation, bounded repricing, execution evidence	Transaction-cost analysis and execution-policy optimization
Journal	Auto-generated trade record, notes, fees, setup attribution	MAE/MFE, behavior analysis, decision-quality reviews
Alerts	Action inbox, severity, acknowledgment and channel settings	Escalation policies, webhook integration, event correlation
Operations	Connectivity, data freshness, restart state, kill controls	Incident replay, drift reports, disaster recovery
Settings	Secrets, account capabilities, risk policies, model budgets	RBAC, tenant isolation, audit retention and licensing

An initial release must be excellent at its supported instruments. It should clearly identify unsupported adjusted contracts, actions and feeds. A visible unavailable feature with a specific reason is better than silently producing an inaccurate estimate.

5. Modes, permissions, and durable authorization

5.1 Modes

- **Read-only:** real account observation; no write credential accessible to the application role.
- **Sandbox:** broker certification orders; isolated state and credentials; no performance claims.
- **Paper:** real or replayed licensed market data, internal simulated orders, isolated paper ledger.
- **Shadow:** real account state and real timing, intent generation only, no broker routing.

- **Supervised live:** user approves a specific reviewed intent; authorization expires when payload, risk, account, or freshness changes.
- **Bounded live:** a user-approved strategy version can act within its policy and capital limits. No per-trade prompt is needed inside the approved envelope.
- **Manage existing only:** opening exposure is blocked; permitted exits/hedges follow a declared management policy.

Changing a mode is a command with a resulting state and audit event. An UI label change alone cannot alter backend permissions. Enabling live trading requires a configured account, matching verified capability, deployed risk policy, tested management path, and an explicit activation command. This blueprint authorizes planning only; no live setup is performed now.

5.2 Authorization object

Store authorization ID, user, account, environment, strategy artifact hash, risk policy hash, approved instruments/products, maximum aggregate budget, maximum per-trade size, schedule, expiry, manager version, exit permissions, and revocation state. Bind the proposal payload and execution policy hash to single-trade approvals. A changed limit price beyond the reviewed range requires a new intent or prior authorized price corridor.

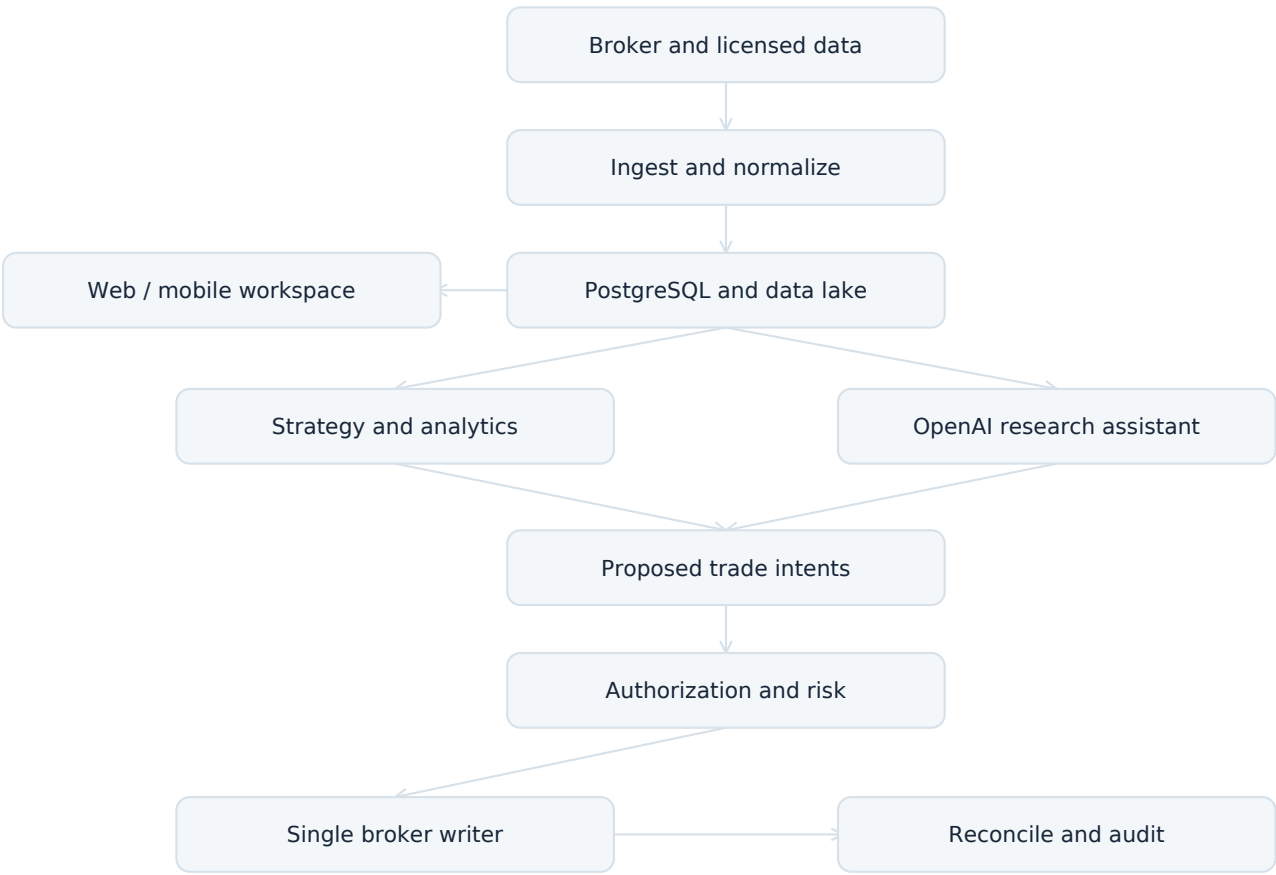
Capability levels: view account, edit drafts, run simulations, approve intent, activate strategy, pause entries, cancel entry orders, manage positions, export records, manage secrets, change global risk limits. In personal mode the same user can hold every capability, but application worker identities cannot.

5.3 Emergency controls

Provide separate actions: pause new entries; cancel working entry orders; cancel selected/all eligible orders; manage existing only; prepare a flatten plan; execute an approved flatten plan. Do not label all five with an ambiguous kill button. A pause should not cancel protective exits. Flattening options requires a fresh portfolio snapshot and ordered plan because removing protection can raise risk or buying-power requirements. Surface positions that cannot be closed and keep the incident open. A broker application remains the independent control path.

6. Recommended architecture

Start with a modular backend and a few isolated worker roles, not a fleet of microservices. Shared domain contracts and tests are more valuable than premature distribution.



Broker fills return through reconciliation. The model does not hold execution authority.

6.1 Technology choices

Layer	Proposed choice	Reason and boundary
Frontend	React, TypeScript, accessible component primitives, client query/cache layer	Rich tables, chart coordination, typed API contracts
API	Python with FastAPI/Pydantic-style typed validation	Async feeds and a strong quantitative ecosystem
Authoritative state	PostgreSQL with migrations	Transactions, unique constraints, durable outbox and audit
Hot cache	Redis only when needed	Quote fan-out and ephemeral cache; never sole order truth
Research storage	Parquet object store and DuckDB	Reproducible, subset-oriented historical analysis
Quant layer	NumPy/Pandas or Polars, vetted option-pricing implementation	Validated numeric routines; no LLM arithmetic authority
Research engine	LEAN proof of concept, native tasty backtester adapter, custom narrow replay as required	Pick after parity tests; adapters preserve result provenance
Model interface	OpenAI Responses API with typed functions and structured output	Controlled retrieval, proposals and explainable summaries
Background work	Durable DB-backed job queue initially; managed workflow engine only if complexity warrants	Schedules, retries, recovery and idempotent read tasks
Deployment	Containers on a managed VM/service with PostgreSQL, secrets manager, object storage	Small operational footprint with clear isolation
Observability	Structured logs, distributed trace IDs, metrics and external heartbeat	Detect missing workers even if the app fails

Use current supported versions when implementing, pin dependency lockfiles, and validate official documentation at that time. Do not decide a model by an unverified marketing name; use a model registry and compare task-specific quality, latency, structured-output adherence, and costs.

6.2 Module boundaries

Identity and policy: users, sessions, capabilities, account access, approvals, strategy activation.

Broker adapter: token management, reads, submissions, cancels, replacements, account stream.

Market data: instruments, streams, snapshots, licensed history, calendars. **Quant:** Greeks, payoff, scenario repricing, indicators, exposure and costs. **Strategy:** versioned definitions, features, signal evaluation, opportunity ranking. **Risk:** policy checks, reservations, portfolio aggregation, stress and incident state. **Execution:** immutable intent preparation, lifecycle, controlled writer, fill assembly.

Portfolio: ledger, lots, reconciled snapshots, strategy attribution. **AI:** bounded tools, evidence packs, schemas, evaluation and cost budgets. **Research:** experiments, datasets, simulations, replay and promotion gates. **Communication:** alert state, channels, summaries and acknowledgment.

Operations: health, recovery, backup and drift.

6.3 Processes and ownership

API/UI worker serves reads and commands. Ingestion worker owns feed connections and durable raw append. Strategy workers read normalized state and create intents. A single execution worker for an account owns brokerage mutations. AI workers run outside the order-critical path. Research workers are resource-limited and cannot access live write credentials. Notification workers consume durable outbox events.

One database can initially serve all modules, with separately scoped roles. Every worker command includes account/environment. Replace hard-coded cross-module calls with interfaces only where necessary. Keep a transactional core; introduce a broker/event bus later only if measured scale or isolation needs justify it.

6.4 Event reliability

Use an append-only event envelope with event ID, schema version, type, aggregate ID, aggregate revision, account/environment, occurred time, received time, correlation ID, causation ID, source, and payload. Financial commands/events live in PostgreSQL; high-volume raw market observations live in partitioned files with indexed decision snapshots. Write state changes and outbox messages in the same transaction.

Consumers process at least once and deduplicate by event ID. Ordering is guaranteed only within our recorded aggregate sequence, not across every external feed. Timestamps alone are not a total order. Do not claim end-to-end exactly-once brokerage effects: the external broker does not enforce our local uniqueness constraints.

7. Data acquisition and quality system

7.1 Source plan

Data	Primary route	Complement/fallback	Storage/use
Accounts, orders, positions, buying power	tastytrade REST and account stream	REST reconciliation and broker UI	Authoritative broker snapshots and normalized ledger
Live equities/options quotes and Greeks	Entitled tastytrade DXLink stream	Licensed independent feed if operationally justified	Latest validated quote cache plus decision evidence
Historical equities	Selected licensed vendor dataset	Broker candles for supported exploratory tasks	Raw and adjusted bars with corporate-action versions
Historical option quotes	Evaluate Massive versus Databento on the target universe	Supported broker backtester as a comparison engine	Contract definitions, quotes, costs and data manifest
Option contract definitions	Broker instrument/chain endpoints	OCC adjustment evidence/vendor metadata	Effective-dated instrument master
Earnings and dividends	Broker metrics plus licensed event calendar	Issuer releases/filings for confirmation	Timestamped event records, confidence and revisions
Company fundamentals	SEC submissions and XBRL APIs	Licensed normalized fundamentals	Filing accession, publication timestamp and units
Macro and rates	FRED/ALFRED and appropriate market feeds	Official calendars for releases	Observation vintage, publication and availability times
News and transcripts	Licensed provider selected during discovery	Company investor relations and SEC filings	Permitted content, IDs, timestamps, source links
Halts/sessions/corporate actions	Broker status/calendar and licensed feed	Exchange/OCC verification where required	Risk gate metadata and replay events

Massive documents historical option quotes; Databento documents OPRA live/historical data and NBBO reconstruction. Evaluate their actual entitlement, adjusted-contract coverage, historical depth and price on a small representative sample. Do not scrape consumer platforms or treat a free/delayed quote as executable. [S15, S16]

SEC supplies unauthenticated submissions/XBRL data, and FRED exposes economic series and vintage-related APIs. Run ingestion on the backend, with service-appropriate identification, cache, throttling, and use rights. [S17, S18]

7.2 Minimum data needed before a first strategy

Get instrument metadata, live bid/ask and size, chain definitions, effective multiplier/deliverables, account restrictions, buying-power data, holiday/session calendar, and event dates. Add quote history before claiming an options strategy has been backtested credibly. For a stock-only baseline, adjusted bar history and realistic execution assumptions can support the first experiment.

Start with a focused liquid universe. Broader market scanning can use inexpensive bar snapshots to identify candidates, followed by quote and option subscriptions only for survivors. Do not subscribe to every strike of every equity continuously by default.

7.3 Canonical timestamps and provenance

Represent event time, source publication time, provider time, our ingestion time, and feature availability time separately. Store UTC; display exchange time and user-local time with clear labels. Instrument sessions use their exchange calendar. Market timestamps need high resolution; database timestampz alone is not enough if the vendor exposes nanoseconds, so retain original integer timestamps.

Decision inputs are immutable snapshots referenced by hash. Each snapshot contains instrument definition revision, latest eligible quotes per leg, underlying price, event/calendar version, rates/dividends assumptions, computed features, account snapshot revision, and data-quality results. Every feature has a calculation version and maximum permissible age.

7.4 Quote validation

Check numeric validity, bid/ask non-negativity, $\text{bid} \leq \text{ask}$ where the feed semantics require it, size validity and units, instrument match, source latency, exchange session, halt state, and timestamp plausibility. Locked/crossed markets are a flagged condition; define which orders may proceed, if any. A quiet option may legitimately have no recent event, so distinguish last-event age from transport health and verified snapshot freshness. Missing bid does not mean an option can be sold at its last trade price.

Classify data: valid, valid-but-illiquid, stale, incomplete, suspect, unavailable. A fallback price is labeled with source and confidence; it must not automatically qualify for routing. Price differences across providers may reflect timestamps/venue coverage. Align time before declaring a discrepancy.

7.5 Refresh policy examples

Use push updates for prices and accounts. Recompute small position exposures on relevant data events with short debounce; batch portfolio scenarios to a slower cadence. Reconcile active account state every configurable 30–60 seconds initially, on stream reconnect, around order anomalies, and before material commands; tune based on API limits and measured load. Chain definitions can be cached intraday with explicit refresh triggers for expiry/listings. Fundamentals and calendars update on source events/daily jobs. These cadences are design defaults, not broker service guarantees.

7.6 Storage and licensing

Retain permitted raw data with provider, dataset, schema and entitlement version. Partition historical objects by dataset/date/underlying or an appropriate cardinality-controlled key; do not create tiny files per event. Compact batches, checksum them, and store a reproducible manifest. Keep only quote slices needed for the universe and research horizon unless a broader license and budget justify full-feed capture. Options data volumes can overwhelm a retail project.

Persist decision evidence under the vendor's storage and derived-data rights. Some licenses distinguish display, non-display, redistribution, and AI use. Record each dataset's allowed consumers, retention, user class and commercial restrictions. Broker availability is not automatically permission to redistribute or upload all raw data to a model provider.

7.7 Point-in-time correctness

Research must use the instrument universe, delisted securities, corporate actions, financial statements, event schedules, and macro vintages available at each historical decision time. Today's chain cannot be retroactively reused as yesterday's universe. Open interest may be reported on a delayed schedule; make its availability explicit. Prevent using a complete day's high/low or final candle before it closes. Keep revisions rather than overwriting their original timestamps.

Historical LLM tests have an additional leakage problem: a modern model may already know later outcomes. Data timestamp controls alone do not remove training-set knowledge. Limit historical AI extraction to grounded facts, isolate evaluation datasets, and rely heavily on prospective paper/shadow

evaluation for AI-dependent selection.

8. Instrument and numerical model

8.1 Instrument master

Store stable internal instrument ID, type, display symbol, broker symbol, vendor symbols, streamer symbol, underlying ID, currency, exchange/calendar, tick rules, quantity steps, option right/strike, expiration date/time, last trading time, exercise style, settlement style, multiplier, deliverables, adjusted flag, contract revision and effective interval. An option's expiration date alone does not determine last trading time or settlement mechanics.

Ordinary U.S. equity option examples often use 100 shares per contract, but adjusted contracts require exact metadata. Model share/cash/other deliverable baskets separately from a numerical premium multiplier. Unsupported baskets are blocked from automated trading. Index cash settlement is different from stock delivery; do not apply equity assignment logic to every option. [S19]

8.2 Numerical conventions

Use Decimal or fixed-point for money, prices, strikes, quantities and broker payloads; floating point is acceptable in validated analytics with explicit tolerances. Do not use binary floating point to decide compliance with a price tick. Serialize broker prices according to the schema. Currency sign, position direction and premium debit/credit are separate concepts.

Normalize Greek units before aggregation: option delta per underlying price unit; gamma per underlying price unit squared; theta per day; vega per one volatility percentage point or explicitly converted unit. Vendor unit definitions require contract tests. Keep the raw units and normalized units both. Greeks can be missing or unstable, especially near expiry and for illiquid strikes.

8.3 Required calculations

- Position market value and hypothetical liquidation value with explicit bid/ask assumptions.
- Realized/unrealized P/L including commissions and accrued financing; separate broker values from internally computed estimates.
- Stock/option expiry payoff with fees and exact contract terms.
- Before-expiry scenario valuation using a declared pricing model, rate curve, dividend assumptions and volatility surface.
- Cash flows, cost basis, break-even points and extrema for supported strategies.
- Delta-equivalent shares, dollar delta, normalized SPY-equivalent beta exposure, gamma, theta and vega.
- IV rank and percentile with stated lookback, IV definition, missing-data policy and annualization convention.
- Historical/realized volatility, ATR, moving averages, momentum, relative strength, volume and liquidity measures.
- Expected move as a model-derived estimate, not a statistical guarantee; disclose whether using ATM straddle or volatility formula.
- Scenario loss, liquidity-adjusted liquidation cost, margin headroom and capital tied up under assignment.

For ordinary unadjusted shares/options, signed dollar delta is approximately $\text{quantity} \times \text{multiplier} \times \text{delta} \times \text{underlying_price}$; for stock, multiplier and delta are 1. Raw share deltas across unrelated tickers are not directly comparable. Convert portfolio beta exposure using a benchmark and a documented estimation window. Do not sum dollar gamma or vega until units and shocks are consistent.

8.4 Worked sizing example

For a standard \$5-wide credit vertical collecting \$1.20 with a 100 premium multiplier, terminal economic loss before costs is $(5.00 - 1.20) \times 100 = \380 per spread. A \$100 per-trade limit allows zero spreads, even if the trade feels conservative or has a high model probability of profit. Include anticipated fees and closing cost before sizing.

For a standard debit spread paid \$0.80, terminal loss before costs is \$80. With a \$100 budget it might allow one, subject to liquidity, costs, assignment, portfolio exposure and account constraints.

Mathematical expiry loss is conditional on intended legs remaining intact and correct settlement; it is not a hard upper bound on every operational outcome.

Formula: `allowed_qty = floor(min(trade_budget / conservative_unit_risk, sleeve_remaining / unit_risk, global_remaining / unit_risk, broker_capacity / unit_bp, liquidity_capacity))`, with instrument quantity steps and conservative cost buffers. For stocks, distinguish stop-distance planned risk from a separate gap/stress risk; price gaps can exceed the stop budget.

9. OpenAI integration

9.1 Role of the assistant

The assistant should explain the portfolio, summarize licensed/primary research, maintain a thesis, compare approved structures, build draft strategy definitions, identify inconsistent plans, describe alerts, and answer questions using our own trusted tools. It may produce structured trade proposals and ranked candidate explanations. It cannot bypass authorization, change risk policy, fabricate prices, or gain write credentials.

Use OpenAI function calling for controlled application tools and structured output for schema-constrained proposals. Schema adherence does not prove factual truth, price accuracy, mathematical correctness, or trading edge. Validate all outputs server-side. [S20, S21]

9.2 Tool catalog

Read tools: `get_account_snapshot`, `get_portfolio_exposure`, `get_position_detail`, `get_market_snapshot`, `get_options_chain_slice`, `get_contract_metadata`, `get_event_calendar`, `search_research_evidence`, `get_strategy_version`, `get_backtest_summary`, `get_execution_timeline`, `compare_hypothetical_portfolios`.

Draft tools: `create_trade_proposal`, `create_strategy_draft`, `request_backtest`, `create_alert_draft`, `append_journal_note`. These create reviewable application objects, not broker orders. Tools require account/environment and server-resolved IDs; the model cannot substitute arbitrary account numbers or URLs.

No general-purpose `place_order` tool in the conversational assistant. In bounded automation, a trusted strategy runner can consume a validated AI proposal under a preapproved policy. The authorization/risk engine and controlled writer still decide whether it becomes an executable intent. The optional broker MCP server is configured read-only and explicit environment initially. [S10]

9.3 Context pack

Construct a small task-specific pack: question, user goals/policy, account snapshot revision, relevant position/underlying slice, recent data timestamps, contract metadata, specific source excerpts with evidence IDs, known missing data, allowed strategy universe, supported tools and budget. Never transmit a full account identifier, credentials, unnecessary personally identifying information, or an entire option chain when a slice suffices.

Every factual claim from the assistant carries an evidence ID or is explicitly a hypothesis. Numeric prices and risks in the final response come from tool outputs, not model recall. The assistant must say when evidence is unavailable and allow abstention. Its self-reported confidence is not a calibrated

probability of profit.

9.4 Proposal contract

A proposal contains ID, creation/expiry time, account/environment, strategy version reference, instrument IDs, intent type, thesis, evidence IDs, invalidation condition, candidate leg ratios, desired price corridor, intended holding window, event exclusions, management-policy reference, and missing-data flags. The quant/risk layer enriches it with verified symbols, executable quantity, cost estimate, scenario results and policy decision. The model cannot set the final account risk authority.

Require strict JSON schemas with all object properties declared, required fields, and `additionalProperties: false`; represent genuinely optional values with null where required by the API's schema subset. Pin schema/prompt versions and reject unsupported fields. Process refusal, incomplete output, timeout, malformed evidence IDs, and tool errors as explicit results. [S20]

9.5 Model routing and budgets

Small validated model/task path for source classification and simple extraction; higher-quality reasoning path for a portfolio explanation or strategy draft; separate embedding/search path if justified. All model choices are configuration, evaluated on our cases, and pinned per strategy/evaluation version. Shadow a new model before enabling it in an AI-dependent live strategy.

Do not call the model on every quote. Trigger on selected events, scheduled research, user queries, and a meaningful change in context. Cache analyses by evidence/input hash. Batch offline labeling where supported. Track tokens, latency, tool calls, failure, cost and quality by workflow. Set monthly/day budgets and per-workflow call caps; exhausting the model budget pauses AI-dependent new work while deterministic monitoring continues.

9.6 Data security and prompt injection

Treat news, filings, web pages and user uploads as untrusted evidence, never instructions. Keep system policy and capabilities outside retrieved text. Escape content in the UI and prohibit arbitrary network/shell operations in model tools. Restrict query scope to the user's authorized account. Use retention-minimizing API configuration where appropriate, while acknowledging that `store: false` alone is not a universal zero-retention guarantee across API features and processing. [S22]

9.7 Evaluation set

Include grounded versus fabricated catalysts, stale quote refusal, unsupported contract refusal, spread-direction correction, wrong account attempts, prompt injection in an earnings release, missing-data abstention, conflicting research, math pulled from quant tools, impossible payoff claims, and safe behavior during tool outage. Evaluate factual support, correct tool selection, schema conformance, abstention, unauthorized action rate, latency and cost. Target zero unauthorized mutations in the adversarial suite.

10. Strategy system

10.1 Strategy taxonomy

Support templates for long equity/ETF allocation, momentum/relative strength, trend pullback, bounded mean reversion, pairs/factor-neutral ideas, covered calls, cash-secured puts/wheel, debit verticals, credit verticals, iron condors, butterflies, calendars/diagonals, collars/protective puts, long-volatility straddles/strangles, and hedge overlays. Templates are starting definitions, not endorsements or evidence of an edge.

Treat leveraged ETFs, 0DTE, earnings events, short stock and uncovered options as separate capabilities with stronger data/management/research gates. A covered call still has stock downside. A

cash-secured put still has equity-like downside. Calendars and diagonals have expiry/assignment complexity. Short premium has tail and volatility risk even with a high win rate. Long options may lose their entire premium. Risk labels describe actual exposures and scenarios, not just structure names.

10.2 Immutable strategy definition

Fields: strategy ID and version, owner, instrument universe, selection schedule, warm-up requirements, feature definitions/versions, entry rule AST, contract selection, position sizing, portfolio conflict policy, execution policy, exit rule AST, assignment/expiry policy, event exclusions, maximum hold duration, allowed modes, risk policy, capital sleeve, and dataset requirements.

Use a typed rule AST such as `all`, `any`, `not`, numeric comparisons, crossovers, schedule windows, state predicates, and portfolio predicates. Resolve every feature name in a registry with data dependencies and unit types. No arbitrary Python expression string in a production visual strategy. Research code extensions run in restricted processes and are versioned artifacts.

10.3 Rule evaluation

The evaluator returns `PASS`, `FAIL`, or `UNKNOWN` plus input/evidence details. `UNKNOWN` blocks entry by default. Exit policies specify whether a degraded-data exception is allowed and what bounded action it permits. Each evaluation creates a compact decision record even when no trade is selected. Explain the first decisive failure and allow drill-down to all predicates.

Signals are candidates, not orders. Rank them using a documented score including expected cost, liquidity, forecast uncertainty, portfolio risk contribution, event exposure and capacity. Calibrate statistical probabilities against realized outcomes; separate model-derived risk-neutral probabilities from subjective return forecasts. A 0.30 option delta is not an assured 30% probability of loss.

10.4 Lifecycle and promotion

Draft → static validation → historical research → locked candidate → live paper → shadow → supervised small live → bounded live → paused/manage-only → retired. Versions do not mutate after promotion. New parameters create a new version with a comparative research run. Exit management can continue under the prior version for existing positions while entries use a new approved one; record that split explicitly.

Every promotion has a reproducibility manifest and operational evidence. Do not promote based solely on a high Sharpe ratio, profit factor or recent P/L. A strategy can pass engineering gates while failing research gates. Retirement preserves history and stops new entries; it does not abandon open trades.

10.5 Different risk sleeves

Allow sleeves for Core, Systematic, Opportunistic, and Experimental, with user-defined dollar caps and permitted structures. Use virtual allocation accounting within the broker account; sleeves are not truly isolated brokerage accounts. Global concentration, cash and margin rules always override individual budgets.

For a \$10,000 illustrative configuration, one could leave \$3,000 cash reserve, cap Core at \$4,000, Systematic at \$2,000, and Experimental at \$1,000. That is a budgeting example, not an allocation recommendation. Do not fund every sleeve by default. Mark-to-market losses lower available budgets; winning streaks do not automatically raise authorization caps.

10.6 Strategy conflict management

Define underlying-level ownership and conflict policies: exclusive, share direction, allow hedging, or manual review. Two bots may otherwise create opposing positions or each consume the same buying power. Evaluate portfolio state plus pending reservations atomically. Net broker positions can merge quantities from multiple virtual strategies; attribution must track virtual lots without claiming separately closeable broker inventory.

Any external order/position from the broker UI enters an Unassigned bucket. Portfolio risk includes it immediately. Importing it into a strategy requires a reviewed management policy. Never have a bot close a manual holding merely because its symbol matches a signal.

11. Execution engine and order lifecycle

11.1 Trade pipeline

1. Receive an immutable candidate with account/environment and a strategy artifact reference.
2. Deduplicate the intended decision by strategy version, signal key and decision window.
3. Resolve exact instruments and quantities from current metadata; verify product/account support.
4. Assemble a market snapshot and account revision; validate data quality and calendar.
5. Quant engine computes costs, payoff, portfolio impact, margin request and sizing.
6. Authorization and global risk gate check the exact payload and execution policy.
7. Reserve risk/capital in a transaction across all running strategies.
8. Call broker dry-run; persist the raw response and warnings. A warning may require user review or a predefined policy; never silently suppress it.
9. Recheck quote/account/policy versions and available authorization. If meaningful inputs changed, regenerate/revalidate.
10. Persist the prepared payload, payload hash, unique correlation identifier, writer claim and durable audit event before making the network call.
11. Writer submits once and records the response or an UNKNOWN outcome.
12. Stream and REST reconciliation assemble fills, commissions and resulting positions.
13. Transition reservations into actual exposure, retaining unresolved residual reservations.
14. Activate the management policy for filled quantity; do not wait for a planned full order if a partial execution already created risk.
15. Emit notifications and a journal event with links to the evidence timeline.

11.2 Internal state machine

Maintain separate internal intent state, raw broker status, filled quantities by leg, management state and reconciliation state. Broker labels alone are not our lifecycle truth.

From	To	Trigger / meaning
[*]	Proposed	Validated lifecycle transition
Proposed	Validated	Validated lifecycle transition
Proposed	Blocked	Validated lifecycle transition
Validated	Authorized	Validated lifecycle transition

From	To	Trigger / meaning
Authorized	Reserved	Validated lifecycle transition
Reserved	Prepared	Validated lifecycle transition
Prepared	Submitting	Validated lifecycle transition
Submitting	Working	acknowledged
Submitting	Unknown	ambiguous outcome
Submitting	Rejected	confirmed rejection
Unknown	Working	found at broker
Unknown	Review	cannot establish outcome
Working	PartiallyFilled	Validated lifecycle transition
Working	Filled	Validated lifecycle transition
Working	CancelPending	Validated lifecycle transition
PartiallyFilled	Filled	Validated lifecycle transition
PartiallyFilled	CancelPending	Validated lifecycle transition
CancelPending	Cancelled	confirmed
CancelPending	Filled	execution wins race
Filled	Managed	Validated lifecycle transition
Cancelled	Reconciled	Validated lifecycle transition
Rejected	Reconciled	Validated lifecycle transition
Managed	Closed	Validated lifecycle transition
Closed	Reconciled	Validated lifecycle transition

A broker may report a terminal filled state before all execution records are retrieved. Track expected versus observed fills and reconcile again; do not invent an execution. Partial fills are inferred from quantities even if the broker does not expose a separate partial-fill status. [S09]

11.3 Duplicate prevention and ambiguity

Persist our unique intent before submission, with unique local correlation and intent constraints. The broker correlation field is not a guaranteed idempotency mechanism. On timeout or unclear 5xx, keep risk reserved, inspect order history and live state with pagination, examine fills/transactions/positions, and reconcile for a bounded window. A single empty search is not proof that no order exists because visibility may lag. If ambiguity remains, block equivalent new orders and require explicit incident resolution. [S07]

Exactly-once is a goal for local command acceptance, not a promise about external exchange effects. A writer crash after submit and before recording the response is an expected case. Restart recovery always reconciles that intent before continuing.

11.4 Single account writer and failover

Use one account writer with lease identity and monotonically increasing fencing token stored in the database. Only the broker gateway holds write credentials. Validate the current lease/fence immediately before each network command. A local fence cannot revoke an already in-flight broker request; a leader handover therefore freezes new submissions until old requests are reconciled and the previous writer is stopped or isolated. Do not automatically create a second writer merely because a heartbeat expired. Prefer recoverable downtime to overlapping live order writers.

11.5 Multi-leg orders

For supported spreads, route one net-price combo order, not independent leg orders. Current guides describe up to four option legs; validate distinct symbols, ratios, quantities, net debit/credit, account approval and contract terms. Stock-plus-option and more complex structures may require staged execution or unsupported-state review. Do not claim atomic execution across separate broker orders. [S08]

Residual fills or legging create explicit exposure and a recovery plan. Preserve protective long legs while closing shorts where appropriate; compare account-level risk before and after any close. Rolling is a linked close-and-open economic decision, not merely postponing a loss. Show realized P/L of the old position and incremental risk/cost of the new one.

11.6 Price policy

Initial limit is chosen from a declared corridor using synchronized quotes. A midpoint is a reference, not an executable promise. For entry, favor bounded limit orders. Configure tick-correct increments, minimum time between replaces, maximum attempts, maximum elapsed time and an absolute worst price. Repricing cannot enlarge quantity or exceed approved debit/reduce credit beyond authority.

For risk exits, use a separately authorized urgency policy: patient limit, more aggressive marketable limit within a cap, then alert/escalation. Options market orders are disabled by default. A cap can leave an order unfilled; make that state visible. Stock market/stop orders may be available when supported and explicitly authorized, with gap/slippage limits understood.

Cancel/replace rereads status and remaining quantities, validates the edit, then follows the broker's supported operation. Never treat cancel-requested as cancelled. Never reserve the original and replacement as independent risk if the broker guarantees contingent replacement, but do not release the original until status/fill evidence supports it. Complex orders need their own tested behavior. [S08, S09]

11.7 Management engine

Every position has management owner, latest policy version, trigger basis, profit target, loss/invalidation trigger, hold limit, expiry handling, event policy, hedge permissions, and desired action state. Trigger basis can be option net liquidation estimate, underlying price, portfolio exposure, time, event or thesis invalidation. Price triggers require valid quotes and noise/debounce rules to prevent acting on one suspect mark.

Broker-resident stops/brackets survive our outage when supported, but their order semantics and partial-fill behavior must be tested. Software exits do not survive backend failure. Show that distinction on the trade review. Do not offer a protection label unless a corresponding order/policy is actually active and observable.

12. Risk policy engine

12.1 Hierarchy

Global user risk → broker account constraints → strategy/sleeve limits → underlying/factor/event concentration → individual trade limits → execution controls. A stricter rule wins. Manual trades cannot bypass global limits. User overrides are separate audited commands and cannot bypass broker restrictions or unresolved order state.

12.2 Checks required before opening exposure

Verify credentials/environment, account ownership and trading status, policy and strategy authorization, portfolio reconciliation, no overlapping UNKNOWN intent, valid quote/contract/calendar, instrument eligibility, position ownership conflicts, liquidity, spread cost, tick/quantity constraints, buying-power headroom, cash settlement, contract loss/stress loss, portfolio delta/vega/gamma, factor/sector concentration, earnings/macro windows, assignment funding, daily loss state, scheduled mode and total pending reservations.

Every check returns ID, severity, PASS/FAIL/UNKNOWN, measured value and unit, threshold, input provenance, policy version, explanation and allowed remedy. Avoid a single opaque risk score that conceals why a trade failed.

12.3 Illustrative initial policy

Control	Example for an initial \$10,000 test account	Meaning
Live automation	Off until activated	Planning does not arm an account
Per-trade modeled loss	\$100 inclusive of expected costs	Reject an indivisible contract that exceeds it
Aggregate new automated risk	\$500	Include working-order reservations and existing bot exposures
Cash reserve	\$3,000	Broker buying power may permit more than we authorize
Daily loss circuit breaker	\$200 relative to a cash-flow-adjusted opening reference	Pause entries and apply predefined management; not a guaranteed loss cap
Max active new bot positions	5	Still subject to global exposure limits
Expiry automation	No new 0DTE entries initially	More evidence/controls required before enabling
Uncovered options	Disabled	Permissions alone are insufficient authority
Earnings holds	Block new exposed entries when calendar is uncertain or inside a declared exclusion window	Long and short structures need separate policies
Leverage/short stock	Disabled initially	Enable as separate capabilities
Option entries	Net-price limit combos	No arbitrary legging by default
Stale/unknown state	Block new risk	Escalate existing risk under its management policy

These are test defaults, not personalized investment advice or optimized thresholds. Configure stock stop risk and worst-case gap stress separately. Low/medium/high profiles are different policies, not a

switch that skips risk analysis.

12.4 Continuous portfolio stress

Scenarios include benchmark $\pm 1/3/5/10\%$, relevant single names $\pm 10/20/40\%$, volatility $\pm 5/10/20$ percentage points, skew/term-structure changes, time roll to near/at expiration, event gaps, correlations increasing toward one, spread widening, delayed exits, and broker margin/financing shocks. Shock magnitudes are user-configurable research assumptions. Reprice nonlinearly for large changes; delta-gamma approximation is only a small-shock diagnostic.

Show current and proposed portfolio results side-by-side, worst scenario, major loss contributors, liquidation assumptions, confidence/coverage, and sensitivity to pricing method. Missing metadata blocks a reliable portfolio result; list unsupported positions rather than silently omit them.

12.5 Statistical risk

Add historical VaR/CVaR, scenario distributions, drawdown limits, loss streak monitoring and forecast calibration as advanced tools. Report observation window, horizon, confidence level, number of samples and nonstationarity limitations. Tail estimates with few events are unstable. Monte Carlo outputs must identify the process and dependence assumptions. A mathematically precise display is not proof the underlying model is true.

12.6 Expiry and assignment

Track cash/share delivery obligations, dividend timing, early assignment likelihood indicators, deep ITM shorts with little extrinsic value, borrowing costs, pin risk, AM/PM settlement, holidays, exercise/do-not-exercise deadlines, and broker liquidation policy. Generate an expiry action list ahead of internal cutoffs, not just at the final minute. Supported long/short legs can exercise or assign asymmetrically and leave stock exposure. High-level spread loss calculations do not remove that operational risk. [S19]

Until exercise/DNE capability is explicitly verified, the automated system's default is to close eligible positions before its conservative internal deadline or flag a required manual broker action. Never guess a universal broker cutoff or silently submit an exercise request via a nonexistent endpoint.

13. Portfolio, ledger, and accounting

13.1 Sources of truth

The broker is authoritative for external balances, positions, transactions, fills and restrictions. Our system is authoritative for strategy attribution, intents, reservations, approvals, policy and evidence. Reconciliation bridges them. A discrepancy must be represented; neither side overwrites the other without explanation.

Keep raw broker objects and normalized tables. Ledger events include trades, commissions/fees, interest, dividends, deposits/withdrawals, transfers, adjustments, expiration, exercise, assignment, splits and corrections. Use explicit debit/credit entries with currency and balanced batch checks. A tax-lot view sits above the ledger; it does not replace it.

13.2 Position grouping

Views: broker position, economic strategy group, virtual allocation lot, underlying, sector/factor, expiration and account. Group IDs have opening, adjustment, roll and closing links. Multiple orders may construct one strategy; one broker position can serve multiple virtual allocations. A grouping change updates attribution history and audit, not original execution records.

Support orphan legs, manual positions, adjusted contracts, partially closed spreads and unresolved fills. These should be prominent in the attention queue, with factual exposure and available actions.

13.3 Performance definitions

Show gross and net realized P/L, unrealized marked P/L, estimated liquidation P/L, fee drag, interest drag, cash flows, drawdown, exposure days, turnover, win rate, average win/loss, expectancy, profit factor, holding duration and benchmark comparison. Net-liq change alone is not strategy return if deposits/withdrawals occurred. Use time-weighted returns for manager/strategy comparison and money-weighted returns for investor cash-flow experience, with clear conventions.

Separate trade expectancy from closed-trade win rate. A wheel strategy's premium ledger must include assigned stock losses and cost of capital. Rolls cannot reset a losing strategy's history. Hedge P/L may be negative while reducing portfolio losses; measure risk effect as well as standalone return.

13.4 Reconciliation

At startup and reconnect: subscribe/buffer events, read positions/balances/orders/transactions, apply a snapshot baseline with timestamps, deduplicate and replay compatible buffered updates, then validate again. Where the provider lacks a reliable sequence watermark, repeat reads until stable and flag residual uncertainty. Periodically compare instrument quantities, executions, cash, fees and buying-power state. Do not promise an atomic cross-endpoint broker snapshot when the API does not supply one.

Unknown manual orders immediately consume conservative risk capacity. On mismatch, pause affected new risk, surface exact quantities/cash and snapshot ages, retain evidence, and propose a repair. Ledger correction is an appended adjustment with reason, never deletion of the original record.

13.5 Tax and recordkeeping tools

Provide lots, realized gain/loss estimates, holding periods, fee/interest exports, wash-sale candidate flags, cross-account watchlists and broker-statement reconciliation. IRS rules can cover options and replacement purchases in other accounts, including IRAs; broker reports may not capture all applicable activity. Mark classifications that require judgment as review items. Do not claim automated tax correctness from matching symbols alone. [S25]

Add year-end exports and CPA-friendly evidence, not automatic tax filings. Product-specific treatment such as section 1256, straddles or trader elections belongs in a reviewed tax configuration. Tax-lot sale instructions must be broker-supported and verified; internal lot selection alone does not instruct the broker.

Cash accounts track settled versus unsettled funds using the applicable settlement calendar. Current general U.S. securities settlement is T+1, but account-specific availability and product rules still require verification. Do not use buying power as a substitute for a settlement ledger. [S26]

14. User information and communication

14.1 Action inbox

One durable inbox contains pending approvals, blocked trades, assignment/expiry actions, unknown orders, reconciliation mismatches, policy breaches and service incidents. Each item has owner, severity, opened time, deadline, account/environment, position/strategy reference, evidence, current action state and next safe action. Merely reading a notification is not resolving the incident.

14.2 Channels

In-app stream is primary. Mobile push is ideal for urgent account events. Email is useful for daily summaries and review reports; optional SMS for emergency escalation if the user chooses it. Signed webhooks allow external dashboards. Slack/Discord integrations are optional features, not configured or used in this task. Store channel settings and opt-in per severity/event category, with quiet hours for nonurgent items.

Urgent alerts bypass quiet hours only under an explicit escalation policy. Suppress repeats using an incident key and escalation timer. A recovered state emits a recovery message with the remaining action, if any. Delivery, receipt and user acknowledgment are different events.

14.3 Message content

Trade fill: what filled, account/mode, legs, quantity, price, fees, resulting position, authorized manager and link. Risk alert: what changed, measured impact, affected exposure, action taken, unresolved state and next action. Blocked candidate: which rule failed, input age and remedy. Expiry item: obligation, deadline, relevant contract/settlement style and permitted actions. Avoid invented certainty or celebratory win-rate messaging.

Sensitive balances and positions should be configurable for lock-screen redaction. Approval links open an authenticated app session and revalidate price/risk; an email response or webhook request cannot directly authorize a stale trade.

14.4 Briefing schedule

Pre-market: system health, overnight changes, positions needing action, catalysts, expected automation schedule and capacity. Intraday: only meaningful change digests and urgent incidents. Post-market: executions, decisions skipped, portfolio changes, costs and unresolved actions. Weekly: net performance, attribution, slippage, forecast calibration, strategy drift and research candidates. Month-end: statements/fees/interest reconciliation and budget review. These are proposed app schedules, not automations created in ChatGPT now.

15. Research and backtesting discipline

15.1 Research ladder

Write a falsifiable hypothesis before tuning. Fix the target universe, horizon, baseline, available data and cost model. Use exploratory vectorized analysis only for screening. Validate promising strategies in an event-driven replay with realistic lifecycle and account constraints. Lock parameters before testing on withheld periods. Use multiple market regimes and walk-forward splits, while accounting for overlapping horizons and data leakage.

Store every experiment, including rejected parameter sets. Multiple testing creates selection bias; the best of thousands of trials may be luck. Report sensitivity around selected parameters, bootstrap uncertainty with appropriate dependence blocks, cost stress, capacity and turnover. Avoid optimizing one headline metric.

15.2 Options execution simulation

Require contemporaneous leg quotes and underlying values, exact contract definitions and availability, tick/lot constraints and event calendars. Buying at ask/selling at bid is a conservative reference, not a universal fill rule. Midpoint fills require empirical justification and a scenario range. A quote touch does not establish queue priority or execution. Trade prints alone cannot justify historical fills at the last price.

A multi-leg theoretical price from individual quotes is not a historical complex-order book. Model combo execution conservatively, including net-spread uncertainty and rejection/nonfill. Track spread width and market size; cap hypothetical fill volume. Simulate cancellations, replacement delay, partial execution, market halts, assignment, expiration, financing and fees where supported. Explicitly label unmodeled mechanics.

15.3 Research reports

Report gross/net returns, CAGR where horizon supports it, max drawdown and duration, benchmark/relative returns, volatility, Sharpe/Sortino with defined annualization, tail losses, expectancy, win/loss distribution, time in market, turnover, fee/interest/slippage drag, capital/margin use, stress outcomes and trade-level logs. Show out-of-sample separately. Tiny samples receive confidence warnings in context, not a fabricated score.

15.4 Promotion evidence

Require deterministic replay reproducibility, schema/data coverage, costs and multiple fill models, no prohibited lookahead, robust neighboring parameters, supported regime/horizon, prospective paper/shadow agreement, functioning management exits, successful restart/recovery, and no unresolved order or ledger incident. Define strategy-specific minimum event counts and test horizons; a month with no relevant market events cannot validate an event strategy.

First small-live gate should assess fills, prices, operational behavior and risk control under an intentionally limited budget. Research edge and operational correctness are separate approvals. A passing paper system can still lose money live.

16. Operations, deployment and incident recovery

16.1 Deployment environments

Development uses recorded fixtures and no live write secrets. Certification uses broker sandbox. Staging uses permitted read-only production data with paper/shadow ledgers. Production uses approved account-bound live workers. Separate database namespaces and secret identities at minimum; separate accounts/projects where practical. A default or missing environment must fail closed for our application.

UI hosting is separate from always-on trading workers. A static page, laptop, serverless request handler or an active ChatGPT conversation cannot by itself guarantee persistent position monitoring. Deploy execution and monitoring on a supported always-on runtime with a managed database, secret service, backup and independent incident channel.

16.2 Service health and initial targets

Measure market/account stream connectivity separately, heartbeat age, quote age, lag by instrument, model latency/cost, queue lag, writer lease, reconciliation age, pending UNKNOWN commands, risk freshness, disk/storage budget and notification success.

Proposed service targets: local pre-trade rule evaluation below 250 ms at p95 for a normal portfolio; UI cached reads below 500 ms p95; critical internal risk-event propagation below 2 seconds p95; alert dispatch attempted within 5 seconds when the notification provider is healthy. External data delivery, broker response, fill and mobile delivery are measured separately and cannot be guaranteed by these targets. Do not put an LLM request inside a millisecond trading loop.

16.3 Failure policies

Failure	New entries	Existing positions	Recovery
OpenAI outage/budget exhausted	Pause AI-dependent entries	Deterministic manager continues	Cache/explain unavailable state; do not fabricate a replacement judgment
Quote feed failure	Block affected opening risk	Existing broker exits continue; software exits alert/degrade under policy	Reconnect, validate snapshots and compare providers before resuming
Account stream failure	Pause or restrict until fresh REST state	Reconcile REST within permitted load	Restore stream, reconcile orders and ledger
Broker timeout after submit	Block equivalent new intent; preserve reservation	Treat outcome as unknown exposure	Inspect history/fills/positions; unresolved incident if ambiguous
Database unavailable	Block all new application commands	Broker-resident orders may still work	Restore DB and reconcile before worker restart
Worker crash	No second writer without recovery	Independent monitor alerts; broker exits may still work	Reconcile in-flight commands and active management policies
Notification outage	Avoid entries whose policy requires successful alerts	Continue manager and retry bounded outbox delivery	Alternate configured channel; unresolved delivery status
Clock skew	Block time-sensitive actions	Conservative monitored mode	Restore clock, invalidate cached decisions
Unexpected assignment	Stop overlapping new exposure	Update actual holdings and risk immediately	Reconcile, fund/close under approved policy or prompt required broker action

16.4 Runbooks

Maintain short actionable runbooks for unknown order, expired/revoked auth, data discrepancy, orphan option leg, unexpected assignment, account call/restriction, queue backlog, corrupted research data and backup restore. Each specifies detection, automated containment, user-visible facts, broker-app checks, permitted remediation, evidence to retain, and criteria to resume.

16.5 Security

Use TLS, authenticated sessions, 2FA/passkeys where available, scoped worker identities, a secrets manager, redaction, least-privilege DB roles, encrypted storage and backups, dependency scanning, explicit outbound allowlists and session revocation. Prevent CSRF on commands and sanitize incoming webhooks. All outbound channel/webhook URLs are configured and validated; do not let a model create an arbitrary exfiltration destination. Separate research execution from live credentials and brokerage network authority.

For a multi-user service add tenant isolation, row-level authorization, independent audit review, broker approval, market-data redistribution rights and appropriate legal/product review. These are launch dependencies for a service handling other people's accounts, not an extra requirement to design a private research tool.

16.6 Backup and recovery

PostgreSQL point-in-time recovery and encrypted object versioning protect ledger and research manifests. Back up configuration hashes and secret recovery procedures separately. Restore into a nonrouting environment, verify ledger and incident states, then reconcile with the broker before any live writer is armed. Never replay an old order outbox automatically after restoring a backup.

17. Verification and release gates

Test high-impact behaviors, not just function signatures. Include property tests for payoff/sizing and reservation invariants, broker contract tests in certification, stream reconnect and duplicate-event tests, execution-race tests, portfolio reconciliation tests, option lifecycle fixtures, and UI workflows. The separate test matrix enumerates actionable cases.

Required invariants: no unapproved order; no order on stale/unknown prohibited inputs; no duplicate local submission for one accepted intent; no risk release while broker outcome is unresolved; cumulative fill quantities are deduplicated; no accidental naked short from a management action; all pending orders/reservations count toward capacity; corrected broker records append evidence; environment identities never mix.

Perform fault injection around the submit acknowledgment boundary, cancel-versus-fill race, token refresh under concurrent reads, worker handover, database transaction failures, wrong symbol encoding, daylight-saving/early-close schedule, manual broker orders, and restored outbox. Verify graceful refusal when contract metadata is unsupported.

Test report records exactly what was exercised, with run ID, engine/adaptor/schema versions, environment and results. A checkbox called tested without evidence is not a release gate.

18. Implementation sequence and estimates

Timing is a planning estimate for one experienced developer, with substantial variation from API access, data rights, strategy scope and research evidence. Best-quality implementation means a reliable core followed by richer features; it does not require every advanced feature before the first usable release.

Stage	Approximate effort	Deliverable	Exit evidence
0. Capability and research spike	1-2 weeks	OAuth read connection, streams, sandbox order lifecycle, LEAN/native backtester comparison, data sample	Exact contract registry and unresolved-gap decision log
1. Read-only portfolio workspace	2-4 weeks	Reconciled portfolio, watchlists, contract details, health and alerts	Accurate manual-trade/fees handling and recovery
2. Trade analytics and supervised paper	3-5 weeks	Chain, builder, payoff/scenario library, risk preview, realistic simulated orders	Numerical fixtures and paper lifecycle consistency
3. Research and automation foundations	3-6 weeks	One stock and one defined-risk option strategy, shared rule evaluator, replay/paper/shadow, journal	Reproducible reports and mode parity
4. Supervised small live	2-4 engineering weeks plus observation time	Approved intents, controlled writer, management and incident runbooks	Live operational evidence at restricted size

Stage	Approximate effort	Deliverable	Exit evidence
5. Bounded automation	3–6 weeks plus strategy-specific observation	Immutable authorization, sleeves, global reservation and monitor controls	Fault-injection/recovery and prospective strategy evidence
6. Advanced workbench	6–12+ weeks in increments	Risk surfaces, optimizer, portfolio allocation, mobile refinement, advanced journal and optional feeds	Feature-specific quality and cost gates

Do not sum engineering weeks into a promise of strategy profitability. A good first read-only release can arrive sooner; credible unattended options trading requires the additional gates and observation.

18.1 First ten concrete implementation tasks

1. Create the typed domain models and environment/account identity model.
2. Implement OAuth TokenManager and a read-only broker adapter.
3. Verify instrument/chain and stream symbol mapping on a small test universe.
4. Ingest account stream and reconcile REST positions, orders and ledger.
5. Build Overview, Portfolio and Operations views with data-age indicators.
6. Implement unadjusted stock/option payoff calculations and sizing fixtures.
7. Build one immutable trade-intent and dry-run review path in certification.
8. Implement UNKNOWN submission handling, single writer, reservations and fault tests.
9. Build realistic paper/replay and one strategy with identical rule evaluation.
10. Add the grounded OpenAI assistant with read/draft tools and evaluations.

Portfolio truth and execution safety precede autonomous AI trade selection. That order avoids spending months polishing an assistant over an unreliable account model.

19. Cost planning and economic viability

Estimate costs by workload rather than assuming a fixed API bill. Track infrastructure, managed database/backups, market-data display/non-display entitlements, historical data downloads/storage, model input/output/cache, news/calendar, notification delivery, commissions, exchange/regulatory fees, borrowing, interest, taxes and engineering maintenance.

Model monthly inference cost as the sum over workflow calls of `input_tokens × input_rate + output_tokens × output_rate + applicable tool/storage cost`, with current vendor rate cards and the proper per-token units. Report research calls separately from live operational calls. Capture actual usage before deciding a model upgrade.

A planning allowance might be \$100–\$400/month for an early always-on personal workspace before expensive history/news and trade costs, but that is an estimate, not a current provider quote. High-volume options data and broad AI research can exceed it substantially. Confirm chosen subscriptions before buying.

For a \$10,000 account, \$200/month is \$2,400/year, or 24% of initial capital before trading costs. A professional-feature custom system can be worthwhile as an engineering/research asset, while being economically unattractive purely for trading such a small account. The app should make this cost drag visible and offer a lean data/model profile. Do not obscure subscriptions behind attractive gross backtest returns.

20. Decisions to make when implementation starts

Choose funded accounts and paper balance, cash/margin regime, initial universe, swing/intraday horizon, acceptable modeled/stress loss, permitted products, earnings/expiry policy, data vendor/entitlements, hosting location, mobile channels, research engine, first two strategies and live authorization path. None is required to read or refine this design.

Recommended first release: reconciled read-only account + useful research + chain/builder + risk preview + realistic paper + one transparent strategy + grounded assistant + operations/alerts. Keep leveraged ETFs, naked options, ODTE and automatic exercise out of initial bounded live scope until separately validated. Each expansion should increase verified capability, not just add a screen.

Screen and interaction specification

Trade Workbench — revision 1, October 6, 2026

The accompanying HTML is an explorable design prototype with synthetic data and local interactions. It has no account connection, OpenAI calls, outgoing messages, persistent backend, or trade routing. Production behavior is specified here and in the blueprint.

1. Application shell

Desktop has a compact left navigation, a top account/environment bar, a central work area, contextual inspectors and a small system footer. Put the current account and mode on every trade-related screen. The central area shows one primary task at a time. Advanced information is available through tabs, expandable rows and inspectors, without disappearing from the underlying risk calculations.

Navigation groups: Today (Overview, Inbox), Markets (Markets, Options, Research), Trade (Trade builder, Portfolio, Orders), Automate (Strategies, Research Lab), Review (Risk Lab, Journal), System (Assistant, Operations, Settings). A command palette searches symbols, positions, strategies, orders and actions. Keyboard commands can navigate or open review; they never submit a live trade without its normal authorization flow.

At 1440 px and wider, show a 200–224 px navigation and a 300–360 px inspector when opened. At 1024 px, navigation is narrower and inspectors overlay or replace a column. On tablet, use compact navigation and stacked analytical panes. At 320–480 px, use a page selector or bottom destinations for Overview, Portfolio, Inbox and More; reveal dense chain/order tables through responsive layouts. Do not hide active risk or environment labels on mobile.

Top bar fields: product, current workspace, account alias and masked ID, currency, execution mode, current session with timezone, data age, connection status, and Pause entries. Keep Cancel orders and Flatten behind a clearly labeled control menu that shows distinct consequences. A pause has its own backend status, progress and result.

Every financial number includes currency or unit, basis, timestamp and an inspection path. Status must be conveyed in text as well as color. Green does not always mean safe; short-option theta and profit are not overall risk classifications.

2. Visual system

Use restrained neutral surfaces, strong legible text, tabular numerals and a single calm accent for navigation/actions. Positive/negative P/L colors are semantic and accompanied by signs. Warning amber means attention; danger red means a blocked action or active incident. Distinguish read-only, paper and live using explicit labels and icon/text cues rather than theme color alone.

Desktop type hierarchy: 24–28 px screen title, 18–20 px key value, 14 px normal content, 12 px secondary labels. Table density is user-configurable. Maintain visible focus, accessible labels, sufficiently large touch targets and usable zoom. Charts have directly labeled axes/units and accessible textual summaries. A compact sparkline is not a replacement for a detailed return report.

Provide light/dark modes, respecting system appearance, and a reduced-motion setting. Avoid flashing quote colors and live-region announcements on every tick. Show only material changes to assistive technology.

3. Shared states and interaction contracts

All screens define first-load, refreshing, populated, empty, stale, partially available, forbidden and disconnected states. First-load uses bounded skeletons; refresh preserves prior data with age labels. Never replace unavailable quotes with zeros. An empty table says why it is empty and provides a relevant next action.

Selection follows a stable instrument ID. Filters and preferences can be saved independently of the current market snapshot. A user cannot accidentally carry a trade ticket from a paper account to live by switching the global account. Switching account/mode invalidates the ticket's approval and requires re-review.

Buttons mutate via typed commands with request IDs, server acknowledgment and observable completion. Disable duplicate commands in progress. After a timeout, show Unknown outcome for broker mutations and inspect/reconcile; do not show a generic Retry submission button.

Dangerous actions explain the concrete account, orders/positions affected and policy. Pausing entries can be fast and reversible. Flattening requires a specific plan because it can close useful hedges and create residual risk. Review is the primary last step for a new supervised trade.

4. Overview

Primary question: what is happening now, what needs action, and is the system operating correctly?

Show account value and cash/available capacity, daily cash-flow-adjusted P/L, authorized automated risk used, active strategy modes, and a compact attention queue. The queue prioritizes unresolved orders/assignment ahead of optional proposals. Include a short portfolio preview and recent execution/decision timeline. Show Next scheduled work with market timezone. Avoid a dashboard of every indicator and a percentage-complete concept.

Main actions: inspect attention item, open portfolio, review proposal, pause entries, inspect system status. Each opens the underlying object rather than a generic report. A stopped strategy can show its management-only positions and why new entries are suspended.

Data: reconciled account snapshot, portfolio risk snapshot, inbox aggregates, worker/mode status and latest decision events. Refresh from server events; risk-dependent actions require an acceptable snapshot age. Acceptance: the user can identify an UNKNOWN order, see whether existing exits remain managed, and navigate to the related evidence within two actions.

5. Markets workspace

Left: saved watchlists or scan results with symbol, price, percentage change, volume/liquidity, data age and event flag. Center: price chart with volume and selected indicators. Below: fundamentals/events or relative performance. Right inspector, when useful: linked research and supported options expirations.

Filters include universe, price, volume, spread, volatility, trend, sector and upcoming event; advanced filters use a typed expression builder. A saved screener records the exact feature definitions. Show how many candidates were removed for missing data rather than merely reporting a result count.

Clicking a symbol synchronizes chart, chain and evidence views, with a consistent symbol-color link if multiple panes exist. Chart cursor information is a historical value, not an executable quote. Drawing/indicator alerts produce draft alert definitions. Trading from the chart opens a ticket, never an immediate account mutation.

Chart preferences: daily/intraday interval, adjusted/unadjusted historical basis, regular/extended session, indicators, visible range, benchmark and timezone. Use a properly licensed chart library with our own data adapter. Crosshair tables show series timestamps, values and missing observations.

Acceptance: changing the selected symbol changes all linked panes; stale input is visible; scan criteria and selected row can be reproduced from a snapshot.

6. Research and thesis workspace

Tabs: Brief, Evidence, Financials, Events and Thesis. A brief begins with sourced facts, recent changes, alternative explanations and unknowns. Each factual statement links to a filing/news item and publication time. Forecasts and hypotheses are visually distinct from facts.

Evidence table: source, title, event/publication time, ingestion time, permitted excerpt, relevant instruments, reliability notes and extraction version. The user's thesis records expected catalyst, expected horizon, invalidation, decision to take no action, and optional comparison to a benchmark. Changes append to thesis history.

Trade action creates a draft based on a thesis; numerical payoff comes from the quant engine. Research approval does not grant trading authority. Source conflicts become review items. If event coverage is incomplete, the product says Calendar coverage incomplete instead of No earnings risk.

Acceptance: every assistant price/financial fact resolves to an input; a later revised filing cannot overwrite the earlier evidence used by a historical trade.

7. Options chain

Top: underlying quote with age, expiry selector, DTE, contract/session metadata, and filters. Chain centered on strikes: calls on left and puts on right for wide screens; on mobile show one right at a time with strike retained. Columns available: bid/ask, spread, size, last-trade age, volume, OI availability date, delta/gamma/theta/vega, IV, intrinsic/extrinsic, and adjusted-contract marker.

Default density exposes bid/ask, delta, IV, volume/OI and strike. Additional columns are configurable. Selection uses explicit Buy/Sell buttons, with action and leg summary visible; clicking a price may prefill a ticket but must not submit it. Long/short legs have both text and color. Adjusted contracts show exact deliverables or an unsupported badge.

An IV view shows skew and term structure with source/method. OI is not falsely shown as live. Estimated probability metrics disclose methodology. An unavailable Greek remains unavailable; it is not silently inferred from an unrelated contract.

Acceptance: selection preserves the exact broker contract ID and effective metadata; the trade builder cannot route a stale or unsupported chain row.

8. Trade builder

Sections: thesis/intent, instrument and leg table, order pricing/quantity, payoff/scenarios, portfolio impact, management plan, preflight and review. On desktop the leg/order pane and analytics pane sit side by side; on mobile the ticket leads, followed by economics and limits.

Leg row fields: broker-resolved symbol, expiry, strike, right, Buy/Sell and Open/Close, ratio/quantity, quote with age, adjusted flag, editable action and remove. Quantity must respect steps and ratios. Changing a leg invalidates earlier risk checks. Template controls include stock, long call/put, debit/credit vertical, covered call, cash-secured put, condor, calendar/diagonal, butterfly and custom supported structure.

Ticket fields: account/mode, net Debit/Credit, limit, tick rule, time in force, execution policy and latest approved price corridor. Show total cost/credit in dollars and number of contracts separately. A premium price such as 0.85 is not the total \$85 contract cost. Quantity is never confused with shares delivered.

Payoff view: expiry curve, break-even points, modeled maximum gain/loss with applicability conditions, fees, scenario cursor, and before-expiry repricing controls for underlying/time/volatility. Before-expiry uses full declared valuation; do not label an expiry-only calculation as a live option forecast. Greeks and scenario tables support detail inspection.

Portfolio impact: current versus after-trade cash, buying-power requirement, modeled risk, worst tested scenario, delta/vega/gamma, concentration and pending reservation. Check results are individually inspectable. Any unavailable scenario coverage is shown.

Management plan: owner, targets, invalidation/stop basis, internal expiry close deadline, event policy, maximum hold, and whether protection is a verified broker order or a software rule. Review is unavailable until required checks pass or declared supervised warnings are accepted.

Acceptance: changing quantity updates all dollar amounts; a \$100 policy rejects an indivisible \$380-risk spread; changing account invalidates the ticket; UI and server give the same reason for a blocked trade.

9. Trade review and approval

Review presents an immutable, concise snapshot: account/mode; action/quantity and all legs; debit/credit and price corridor; total cost including fee estimate; conditional maximum loss and assignment obligation; portfolio impact; specific passed/failed/unknown checks; management owner; event dates; broker dry-run warnings; expiration time of approval; and data timestamps.

Primary action in supervised live is Approve this trade with precise identity. The server approves the payload hash and rechecks at dispatch. Approval succeeds as an application command; it is not the same as a fill. Show Submitted, Working, Partial fill, Unknown, Rejected or Filled as actual lifecycle outcomes.

Automatic mode review focuses on the policy envelope: strategy version/hash, account, structures, universe, schedule, sleeve limit, per-trade and global caps, price/repricing policy, management/exit exceptions and expiry. Primary action is Activate this strategy within these limits. No arbitrary future change inherits that authorization.

10. Portfolio

Top summary distinguishes broker net liquidation, internally computed mark P/L, estimated liquidation P/L, settled cash and margin capacity. Main table groups by economic strategy or underlying with account, owner/mode, quantity, basis, mark, unrealized/realized net P/L, max modeled risk, expiration, event/management status and update age.

Expandable groups show every leg, fill, fee and attribution lot. Filter by account, strategy, underlying, sleeve, expiry, owner, supported/unsupported and manual/automated. A separate exposure lens shows factor/sector/underlying and cannot silently omit unsupported positions.

Actions: inspect, simulate close/roll/hedge, transfer virtual attribution with audit, take manual management ownership, and export. Close opens a reviewed closing intent. Roll comparison shows closing P/L and costs separately from the new position. A position inspector links thesis, risk, management policy, orders, executions, events and journal.

Acceptance: positions opened in the broker app appear promptly in Unassigned; they count toward global risk; a strategy cannot accidentally close another owner's holdings.

11. Risk Lab

Tabs/lenses: Limits, Exposure, Scenarios, Margin, Expiration and Liquidity. Scenarios have current/hypothetical portfolio selectors and assumptions for underlying/factor moves, IV/skew shifts, time, spreads and rates. Initial view is a small scenario table with a dominant payoff/risk visualization, not dozens of opaque gauges.

Results: dollar/percent portfolio impact, affected positions, largest contributors, buying-power headroom, assignment cash, model coverage, timestamp and method. Add/remove/increase/decrease positions in a hypothetical portfolio without changing real holdings. Show current and proposed on the same scale. Historical scenarios specify data dates and applicability.

Limits show actual measurements, thresholds and status with units. A budget ceiling is not an allocation target. A candidate hedge lists risk reduction versus cost and new exposures; reducing delta is not necessarily reducing every risk.

Prototype scenario sliders use an explicitly labeled approximation on synthetic data. Production large-shock analysis requires full nonlinear repricing with supported contract metadata.

Acceptance: a scenario exposes unsupported positions; a closing protective leg can be flagged as risk-increasing; input assumptions and calculation version are exportable.

12. Strategies and visual editor

Strategies list: name, immutable version, account, environment, lifecycle stage, budget used/remaining, next evaluation, managed positions, last decision and reason, incident status and entry mode. Pause entries is separate from Stop management. A paused strategy's managers remain visible.

Detail/editor sections: Universe, Entry, Contract selection, Sizing, Execution, Management, Events, Risk, Schedule, Evidence, Version diff and Promotion. Visual recipes use typed branches with PASS/FAIL/UNKNOWN outputs. A side inspection of a selected branch shows actual values and timestamps for a recent decision.

For entry criteria define conjunction/disjunction, lookback, operator, units and missing-data policy. Contract selection supports expiry interval, target delta/strike offset, width, liquidity and adjusted exclusions. The editor validates dependency availability and current account product support. The code view is a generated typed AST/configuration, not a separate inconsistent implementation.

Editing a live strategy creates a draft revision. Users see semantic changes and affected authorizations. Promote opens an evidence checklist linked to actual runs; authorization is bound to the version. A cloned strategy receives a new ID and zero live authority.

Acceptance: visual/code views produce equivalent AST; UNKNOWN data blocks default entry; changing a parameter cannot mutate an active version invisibly.

13. Research Lab and backtest replay

Experiment setup: frozen strategy/version, universe, dataset manifest, time range, warm-up, initial capital, account rules, commission/interest, fill/slippage scenarios, event/assignment assumptions, holdout partition and benchmark. A cost preview precedes large downloads or runs.

Results: equity/drawdown curves, gross/net/benchmark comparison, costs, statistical uncertainty, regime breakdown, distribution and trade-level logs. Separate training/validation/holdout and paper/live returns. Comparison views use consistent capital, dates, costs and definitions. A broker backtester result is labeled with engine/version and assumptions rather than flattened into our engine's identity.

Replay: play/step through decision events, display the market/account snapshot visible at that time, show rules and order outcomes, and explain blocked decisions. It must not load a current quote into a historical replay. Navigation to an individual trade restores its environment/time context.

Promotion action creates a locked candidate and links required prospective observation. A compelling curve alone cannot activate live trading. Synthetic prototype result curves are marked Synthetic demonstration, never Historical performance.

14. Orders and execution inspector

Blotter columns: environment/account, internal intent, broker ID, strategy version, instrument summary, order type, net price/effect, submitted/fill/remaining quantity, raw broker status, internal state, update time, reserve and incident. Filters distinguish all current active orders from today's history; provider route names do not determine our UI meaning.

Inspector timeline: proposal, market/account snapshot, rule decisions, risk reservation, dry-run, authorization, writer claim, submit attempt, acknowledgment/unknown, broker status updates, fills, costs and reconciliation. A fill records exact quantity, price, timestamp and execution identifier; arrival/decision benchmarks make slippage inspectable.

Actions depend on verified capabilities and current state. Cancel is a requested operation until confirmed. Replace shows new price corridor and renewed checks. Unknown outcome offers Reconcile and Open broker, not Blind retry. Multiple selected cancels report per-order results.

Acceptance: an acknowledgment timeout preserves reservation; cancel-fill race displays actual resulting inventory; reopening the page cannot create another intent/submission.

15. Inbox and notifications

Sort actionable items by severity/deadline. Categories: order ambiguity, risk, expiration/assignment, approval, data/account health, research coverage and informational. Each item includes facts, current action taken, ownership, deadline/timezone and a route to the related object. Distinguish new, acknowledged, action in progress and resolved. Acknowledged is not resolved.

Alert configuration includes thresholds/conditions, event dedupe interval, severity, channel selection, sensitive-field redaction, quiet hours and escalation. Test channel controls send only an explicitly initiated test during actual implementation; this prototype simply changes local settings.

Acceptance: urgent incident recovery does not close a still-unreconciled order; duplicates collapse into one incident with an event history.

16. Journal and reviews

Generate a journal entry from the full trade provenance. Show thesis, intended holding horizon, risk budget, entry/exit decision, fills/costs, management changes, rolls, outcome and benchmark. Add the user's notes and screenshots as attachments with timestamps and retention rights. Editing a note does not alter the initial thesis or executed payload.

Analytics group by strategy version, setup, instrument, time, regime, owner/mode and cost. Include expectancy, R-multiple based on the actual initial defined risk, MAE/MFE, win/loss distributions and process-rule adherence. Missing/highly uncertain risk denominators make R-multiple unavailable. Keep paper, backtest and live statistics separate by default.

Weekly review describes evidence and uncertainty, with candidate changes as drafts. It cannot silently retune a strategy because of a losing streak.

17. Assistant

Conversation is adjacent to an explicit context selector: account/environment, selected position/strategy and evidence timestamp. Example prompts: Explain my current risk; Compare this debit spread with stock; Why was this entry blocked?; What changes if volatility rises?; Draft a strategy using these rules.

Responses include facts, source/evidence links, quant-tool figures, uncertainty/missing inputs, and draft objects. Action chips open a real proposal/risk/strategy object rather than executing an instruction from prose. Retrieved text never changes tool authority. Streaming responses can show progress but no financial action is taken until an authorized application command exists.

Prototype responses are scripted examples, visibly labeled. Production includes model/prompt/schema trace and usage in an expandable audit view, without exposing secrets.

18. Operations

Health view shows quote stream, account stream, writer lease/worker, database, risk computations, reconciliation, model service/budget and notifications. Each status reports observed time and a useful metric. Show in-flight and UNKNOWN commands separately from ordinary queued jobs.

Incident actions: inspect timeline, reconcile account, reauthenticate connection, pause entries, inspect broker, restore monitoring. Restart/reconnect never implies permission to replay financial commands. Simulation controls exist only in developer/paper environments and are clearly separated from production controls.

An independent heartbeat monitor detects total application failure. A green dashboard within a stalled process is insufficient health evidence.

19. Settings and onboarding

Onboarding sequence: create workspace; select paper/read-only mode; securely configure broker connection; discover accounts/products; select data feeds and acknowledge required usage rights; define risk policies; set notifications; validate connection/recovery; create first strategy draft. Live activation is an explicit later action on a concrete policy/account.

Settings sections: profile/timezone, accounts/access, data entitlements and quality policies, risk profiles/sleeves, execution defaults, model workflows/budgets, notifications, storage/export, audit/security and system capabilities. Secret displays show only configured/not configured and last rotation; never reveal the stored value.

Public-product onboarding requires broker third-party approval and separate user authorization. Personal grants cannot be reused as a multi-user login mechanism.

20. Interface implementation acceptance

Cover 320, 390, 768, 1024 and 1440 px widths; light/dark; keyboard-only navigation; 200% text/zoom; no color-only statuses; missing quotes; unavailable event coverage; unknown submit; manual broker position; revoked scope; empty account; multi-account switch; unsupported adjusted option; and quantity rounding. All visible trade actions share the server validation contract. Local estimates are always reconciled with server results before execution.

The prototype demonstrates navigation, local trade sizing/payoff, scenario controls, draft strategy editing, local paper order flow, pause state, inbox acknowledgment and simulated quote disconnection. It deliberately leaves broker connections and live actions unavailable.

Application API and event contracts

Revision 1. These are proposed internal application interfaces. They are not tastytrade endpoints, and their idempotency guarantees do not extend automatically to the broker.

1. Identity and types

All objects have UUID IDs, environment (readonly, sandbox, paper, shadow, live), account ID, UTC timestamps, schema version and revision where mutable. Broker account numbers stay behind the server mapping. Decimal currency/price values are serialized as strings with an explicit currency/unit. Instrument IDs are stable internal keys; broker/vendor symbols are effective-dated mappings.

Read requests require an authenticated user authorized for the account. Commands require the appropriate capability, an application request ID and expected revision. Our own command API accepts Idempotency-Key; the gateway binds it to user, route and request-body hash and returns the same recorded command for a matching replay. Reusing a key with a different payload is a conflict. This mechanism does not mean the external broker deduplicates orders.

2. HTTP inventory

Method/path	Request or query	Result and permission
GET /v1/workspace	Workspace ID	Account aliases, modes, user capabilities, nav/health summary
GET /v1/accounts/{id}/snapshot	Optional revision/as-of	Reconciled balances, positions, restrictions and known discrepancies
POST /v1/accounts/{id}/reconcile	Expected revision, reason	Durable reconcile job; never a blind financial retry
GET /v1/accounts/{id}/capabilities	—	Supported/tested products/actions and unresolved gaps
GET /v1/instruments	Search, type, underlying, cursor	Exact metadata IDs and versions
GET /v1/options/chains/{underlying_id}	Expiry, right, strike/delta slice, snapshot ID	Definition plus quote quality/age; nullable values
POST /v1/market/snapshots	Instrument IDs and required freshness	Immutable market snapshot reference and coverage results
GET /v1/watchlists	Workspace	Saved ordered instrument references
POST /v1/watchlists/{id}/items	IDs and expected revision	New list revision; user edit capability
POST /v1/scans	Typed filter AST and universe	Scan result/job and rejection/missing-data counts
GET /v1/research/evidence	Instrument/topic/as-of/cursor	Licensed evidence with availability timestamps
POST /v1/research/briefs	Context scope and evidence limits	Brief job and citations; no broker mutation
POST /v1/analytics/payoff	Exact legs, entry cash flow, assumptions	Expiry payoff/economic extrema and supported coverage
POST /v1/analytics/scenarios	Portfolio snapshot, hypothetical changes, shocks	Valuation/margin coverage and scenario contributions
POST /v1/trade-proposals	Structured draft	Proposed object; untrusted until quant/risk validated
POST /v1/trade-intents	Proposal/legs, account, snapshot, policy	Immutable intent candidate and validation results
POST /v1/trade-intents/{id}/preflight	Expected revision and price corridor	Broker dry-run, risk checks, reservation status, review expiry
POST /v1/trade-intents/{id}/authorize	Exact payload/policy hashes, expiry	Signed server authorization; approve capability
POST /v1/trade-intents/{id}/dispatch	Authorization ID, expected revision	Durable command ID; controlled writer owns submission
GET /v1/trade-intents/{id}	—	Internal lifecycle, raw broker status, quantities, risk reserve
GET /v1/orders	Account/environment/states/cursor	Active/recent orders and reconciliation state

Method/path	Request or query	Result and permission
POST /v1/orders/{id}/cancel	Expected revision, reason	Cancel command; completion must be observed
POST /v1/orders/{id}/replace-plan	Price/time-in-force, expected revision	Reviewed replacement candidate under broker capabilities
POST /v1/orders/{id}/reconcile	Reason	Inspect recoverable state, not resubmit
GET /v1/positions/{id}	Optional snapshot	Group/legs/lots, ownership, management and evidence
POST /v1/positions/{id}/close-plan	Quantity and execution policy	New closing intent, fresh risk analysis
POST /v1/positions/{id}/roll-plan	Candidate new legs/price	Linked close/open comparison and new authority requirement
POST /v1/positions/{id}/ownership	Owner/policy, expected revision	Audited management transfer; global risk remains
GET /v1/risk/policies	Account/workspace	Versioned policy hierarchy
POST /v1/risk/policies	Draft and expected parent	New policy revision; cannot mutate approved past version
GET /v1/risk/snapshots/{id}	—	Measurements, thresholds, provenance, unsupported coverage
POST /v1/strategies	Typed definition	Draft version/artifact hash
POST /v1/strategies/{id}/versions	Draft definition and predecessor	New immutable artifact
POST /v1/strategy-versions/{id}/validate	Dataset/capability scope	Static validation and missing dependency list
POST /v1/strategy-versions/{id}/promote	Target stage, evidence run IDs	Promotion candidate; live activation separate
POST /v1/strategy-versions/{id}/activate	Account, policy hash, budget, expiry	Explicit mode authorization and activation command
POST /v1/strategies/{id}/pause-entries	Reason, expected revision	Pause entry authority; manager state returned separately
POST /v1/accounts/{id}/pause-entries	Scope/reason	Account-level entry pause
POST /v1/accounts/{id}/flatten-plans	Selected inventory and constraints	Explicit per-position/order plan, unsupported residuals
POST /v1/experiments	Strategy/data/cost/split manifest	Research job with estimated cost and artifact IDs
GET /v1/experiments/{id}	—	Status/results/notices, engine versions and reproducibility
POST /v1/experiments/{id}/cancel	Reason	Cancel research job; no trade action
GET /v1/journal	Mode/strategy/date/cursor	Execution-derived entries with user notes

Method/path	Request or query	Result and permission
POST /v1/journal/{id}/notes	Text/attachments/revision	Appended note history
GET /v1/inbox	Severity/state/account/cursor	Durable incidents and deadlines
POST /v1/inbox/{id}/acknowledge	Expected revision	Acknowledgment only; unresolved action retained
GET /v1/operations/health	Scope	Observed process health and lag metrics
POST /v1/assistant/runs	Question, context scope and limits	Grounded read/draft tool run; no direct submit authority
GET /v1/audit	Object/correlation/time/cursor	Permission-scoped append-only evidence timeline
POST /v1/exports	Format/object scope	Download artifact with account access checks

Use cursor pagination with a stable key and explicit snapshot/as-of when needed. Reject arbitrary caller-provided broker URLs. Long tasks return 202 and a job/command ID; browser polling/streaming follows status. Do not equate HTTP success with a filled trade.

3. Error and command results

Return a stable code, user-readable message, command/object ID, retry class, details and trace ID. Codes include ACCOUNT_SCOPE_DENIED, ENVIRONMENT_MISMATCH, REVISION_CONFLICT, POLICY_BLOCKED, DATA_STALE, DATA_UNKNOWN, CONTRACT_UNSUPPORTED, AUTHORIZATION_EXPIRED, BROKER_REJECTED, BROKER_OUTCOME_UNKNOWN, RECONCILIATION_REQUIRED, and RATE_LIMITED.

HTTP classes: 400 invalid shape; 401 missing app authentication; 403 capability denied; 409 revision/idempotency/policy conflict; 422 domain validation; 429 bounded app rate limiting; 503 service unable to prepare command. For a broker command accepted into our durable queue, an uncertain broker outcome is generally a command state, not an invitation to repeat the HTTP request with a new key.

4. Event stream

GET /v1/events via SSE is enough initially for browser updates; use an authenticated WebSocket if bidirectional subscription control is justified. Events are account/environment scoped and permission filtered. Never forward raw broker tokens/headers to the browser. Coalesce replaceable quote snapshots under backpressure, but never drop financial lifecycle/audit events. Slow clients resynchronize from snapshots and durable event cursors.

Envelope example (application-owned schema):

```
{
  "event_id": "11111111-1111-4111-8111-111111111111",
  "schema_version": 1,
  "type": "execution.outcome_unknown",
  "account_id": "22222222-2222-4222-8222-222222222222",
  "environment": "paper",
  "aggregate_id": "33333333-3333-4333-8333-333333333333",
  "aggregate_revision": 7,
  "occurred_at": "2026-10-06T14:35:10Z",
  "received_at": "2026-10-06T14:35:10.080Z",
  "correlation_id": "44444444-4444-4444-8444-444444444444",
  "causation_id": "55555555-5555-4555-8555-555555555555",
  "source": "broker_adapter",
  "payload": {"intent_id": "33333333-3333-4333-8333-
```

```
333333333333", "risk_reserved_usd": "88.00", "next_action": "reconcile"}
}
```

Topics: market.quality_changed, account.snapshot_reconciled, account.discrepancy_opened, strategy.evaluated, proposal.created, risk.checked, risk.reserved, authorization.issued, execution.prepared, execution.submitted, execution.outcome_unknown, order.status_changed, execution.fill_observed, portfolio.position_changed, management.action_due, expiry.action_due, incident.opened, incident.acknowledged, incident.resolved, notification.dispatch_attempted, and worker.health_changed.

Reconnection: client supplies its durable cursor; server replays retained events if permitted and available, otherwise sends `resync_required` with a fresh snapshot revision. The client never reconstructs account truth only from transient quote events.

5. Writer pseudocode

```
async def dispatch(intent_id, authorization_id):
    # One account-bound broker gateway owns credentials and network mutations.
    prepared = await prepare_with_account_lock(intent_id, authorization_id)
    # Transaction checks account/mode, strategy/policy hashes, data age,
    # ambiguity state and reservations, then records an immutable payload.
    # Reconcile/drain old writer requests before any writer handover.
    await gateway.assert_current_writer(prepared.account_id, prepared.fence)
    await assert_current_dispatch_conditions(prepared)
    await record_submit_started(prepared) # durable before network call
    try:
        result = await gateway.submit_once(prepared.payload)
    except OutcomeMayBeUnknown as exc:
        await record_unknown_preserving_reservation(prepared, exc)
        await schedule_reconciliation(prepared) # never blind POST retry
        return
    await record_acknowledgment_or_confirmed_rejection(prepared, result)
    await schedule_fill_and_account_reconciliation(prepared)
```

The network gap after the last local check cannot be made atomic with a broker REST service. Record the limitations, lease handover policy and ambiguity path explicitly. Outbox retries for notifications are different from order-placement retries.

6. Data schemas and migration ownership

`05_database.sql` is a PostgreSQL starting migration illustrating constraints and durable objects. It is not a full finished ORM or production schema. `06_trade_proposal.schema.json` is a strict draft/proposal schema appropriate for an OpenAI structured-output contract; all values still require semantic validation. `07_strategy.example.yaml` shows a research/paper definition, not an approved live strategy or profitable recommendation.

Financial status transitions occur through domain commands that append events. Append-only audit and evidence roles need database permissions/triggers in the production migration. Keep raw broker JSON for schema drift and forensic review, with secrets removed. Record migration/adaptor/schema hashes alongside research and activation manifests.

7. API test acceptance

Test same idempotency key/same payload; key reused with different payload; cross-account access; wrong environment; stale expected revision; expired payload-bound approval; risk race between two strategies; timeout after broker acceptance; cancel-fill race; stream reconnection with missing cursor; slow UI client; model-created unknown IDs; injection in an evidence document; price decimal/tick correctness; and forbidden capabilities omitted from client responses.

No application response should promise execution success until verified fills are recorded. No local simulation response should be mistaken for a live broker result.

High-impact test matrix and release gates

No runtime broker or OpenAI integration has been executed for this design. These are required implementation tests. Prototype checks are reported in README separately.

ID	Case	Required outcome	Stage
T01	OAuth expiry while concurrent workers read	One refresh per credential set; requests resume without leaking secrets	Integration
T02	Sandbox credentials selected with production URL	Environment mismatch blocks before routing	Integration
T03	Wrong account or user in request	Access denied with no cross-account data	Foundation
T04	Option symbol contains spaces or special characters	Broker-resolved exact symbol; correct URL encoding only in URL	Integration
T05	Contract adjusted deliverable unsupported	Automated proposal cannot proceed; original metadata visible	Analytics
T06	Vega/theta unit mismatch in provider	Unit validation fails instead of incorrect aggregation	Analytics
T07	\$380-risk contract with \$100 budget	Quantity zero and clear indivisible-risk explanation	Risk
T08	Credit/debit direction or action incorrect	Semantic validation and broker preflight reject	Risk
T09	Two strategies reserve the same capacity concurrently	Serial/atomic evaluation; total capacity never double spent	Risk
T10	Working entries plus existing positions exceed limit	New candidate blocked including pending reservations	Risk
T11	Closing long protective leg leaves a short uncovered	Risk-increasing close detected; approval/capability required	Risk
T12	Timeout after broker accepted order	UNKNOWN persists; reserve retained; no blind resubmit	Execution
T13	Writer crashes before recording acknowledgment	Restart reconciles order history/fills/positions before resuming	Execution
T14	Broker search temporarily reports no matching order	Keep ambiguity through visibility window; no empty-read proof	Execution
T15	Same internal dispatch request replayed	Same durable command returned; no new local intent	Execution
T16	Same idempotency key with different payload	Conflict with no dispatch	Execution

ID	Case	Required outcome	Stage
T17	Cancel and fill cross in flight	Actual filled inventory recorded; cancel acknowledgment not assumed	Execution
T18	Partial fills and duplicate execution events	Filled quantity deduplicated; manager starts for actual exposure	Execution
T19	Terminal broker status arrives before fill records	Reconciliation remains open until execution completeness checked	Execution
T20	Old writer request in flight during lease expiry	No concurrent replacement writer; handover reconciles first	Execution
T21	Restore old database/outbox	Financial outbox does not auto-submit; reconcile broker first	Operations
T22	Quote socket alive but a candidate quote is stale	Candidate blocked independently of transport health	Data
T23	Quiet instrument has no new event	Distinguish quiet trading from disconnection; fresh snapshot policy	Data
T24	Bid/ask crossed or suspect price spike	Quality rules prevent false stop/entry triggers	Data
T25	Feed subscription capacity exceeded	Position/exit priorities preserved; research subscriptions shed	Data
T26	Account stream disconnects but quotes continue	Account health degraded; new-risk policy uses fresh reconciliation	Data
T27	Buffer/reconnect lacks a reliable sequence watermark	Stable resnapshot or explicit uncertainty; no invented atomic state	Portfolio
T28	User trades directly in broker app	Position/order appears Unassigned and consumes risk capacity	Portfolio
T29	Assignment changes option into stock holding	Holdings/cash/risk updated and relevant entry conflicts paused	Options
T30	Short option around ex-dividend/low extrinsic	Assignment action flag with exact source/contract and funding check	Options
T31	Expiry on early-close/holiday/AM settlement	Instrument calendar and verified cutoff determine actions	Options
T32	Exercise/DNE API not verified	Manual broker action required or conservative close policy	Options
T33	Broker margin changes after preflight	Fresh dispatch check blocks or rebuilds intent	Risk
T34	Event calendar unknown/revised	Unknown blocks default entry; revisions retained	Data

ID	Case	Required outcome	Stage
T35	Cash proceeds unsettled	No implied settled cash from buying-power number	Portfolio
T36	Roll of losing option	Old realized P/L retained; new position risk/fees separate	Portfolio
T37	Corporate action restates cost/contract	Effective revisions preserved; replay uses then-known metadata	Portfolio
T38	Same trade in paper versus live journal	Different ledgers and performance cohorts	Portfolio
T39	LLM cites nonexistent evidence or stale price	Claim/proposal rejected or corrected by trusted tools	AI
T40	News contains instructions to trade/change limits	Untrusted evidence has no command authority	AI
T41	LLM output is valid JSON but impossible economics	Semantic/quant checks block	AI
T42	AI service fails or hits cost cap	Existing deterministic manager continues; dependent entries stop	AI
T43	New model/prompt revision	Versioned shadow evaluation; no inherited live authority	AI
T44	Historical model already knows future outcome	Leakage risk documented; prospective evaluation required	Research
T45	Today's constituents/chain used in past test	Point-in-time checks fail	Research
T46	Midpoint-only backtest looks profitable	Spread/latency/nonfill/cost stress reported before promotion	Research
T47	Independent legs used as atomic combo execution	Unsupported assumption disclosed; conservative replay comparison	Research
T48	Mass parameter search selects one winner	Full experiment count/holdout and robustness evidence retained	Research
T49	Changed strategy parameters while live	New draft artifact; existing managers remain on approved version	Automation
T50	Pause entries	Entries stopped; protective/management actions preserved	Automation
T51	Flatten selected inventory removes hedge first	Ordered risk-reviewed plan and residual exposure displayed	Automation
T52	Critical notification undelivered	Delivery incident persists; configured escalation and entry policy	Alerts
T53	User acknowledges an unresolved order incident	Acknowledged only; financial incident remains open	Alerts

ID	Case	Required outcome	Stage
T54	Mobile approval opens after price/TTL change	Revalidate; stale authorization cannot dispatch	UI
T55	Switch account/mode with ticket open	Approval invalidated; context visibly changed	UI
T56	Narrow display/keyboard/zoom/dark mode	Labels, units, status and primary actions remain usable	UI

Gates

Read-only release: T01–T04, T22–T28, basic ledger matching, no accessible write credential and user-visible connection ages. **Paper/analytics:** payoff/unit fixtures, T05–T11, T34–T38, realistic fill assumptions and clear simulation labels. **Supervised live:** T12–T21, T29–T33, verified management paths, broker capability evidence and explicit account/payload approval. **Bounded live:** T39–T55, unchanged immutable artifacts, prospective strategy evidence, independent outage alert and rehearsed recovery. **Advanced/multi-user:** tenant isolation, broker verification, license/entitlement checks and feature-specific operational tests.

A failed core gate blocks the affected capability; it does not need to block unrelated read-only tools. Operational gates and return/edge evidence are evaluated separately.

Primary-source research register

Checked October 6, 2026. Sources establish current documented capabilities or design patterns; account entitlements and exact executable contracts still require integration verification. Dates, access, prices, regulatory implementation and API schemas can change. All architecture and proposed policies in this pack are our design recommendations.

ID	Primary source	Use in the design
S01	tastytrade OAuth	Current token flow, environment separation and personal versus third-party apps
S02	Stream market data	DXLink, quote tokens, event fields and broker-provided stream symbols
S03	Stream account updates	Independent account notifications and authentication
S04	Rate limits and backoff	REST backoff versus documented stream caps
S05	Sandbox	Synthetic fills, unavailable market-data routes and reset behavior
S06	Options backtesting	Separate backtester host, coverage discovery and result inspection
S07	Idempotency and retries	Correlation handles do not guarantee broker-side deduplication
S08	Orders reference and order types	Submit, preflight, cancel, edit, replace and complex orders
S09	Order lifecycle	Raw broker states and reconciliation around fills
S10	tastytrade MCP server	Optional self-hosted integration and read-only/environment settings
S11	TradingView features	Linked charts, market exploration and alert patterns
S12	OptionStrat features	Payoff exploration and candidate optimization; direct fetch was restricted, official search content inspected
S13	Option Alpha backtesting and automation testing	Research-to-automation continuity and decision logs
S14	QuantConnect tastytrade integration and backtesting	Evaluate existing brokerage/research engine and paper model
S15	Massive historical option quotes	Historical bid/ask data candidate; verify selected plan rights and coverage
S16	Databento options NBBO example and dataset details	OPRA data and point-in-time symbol/coverage considerations
S17	SEC EDGAR APIs	Filing submissions and XBRL fundamentals
S18	FRED/ALFRED API	Economic series, releases and observation vintages

ID	Primary source	Use in the design
S19	FINRA options	Exercise, assignment and option-specific risks
S20	OpenAI function calling	Typed controlled tools and strict schemas
S21	OpenAI structured outputs	Proposal output contract; semantic correctness still external
S22	OpenAI data controls	Feature-specific retention and data-minimizing design
S23	Interactive Brokers Risk Navigator	Hypothetical portfolios and scenario drill-down
S24	FINRA intraday-margin transition	Avoid a hard-coded universal PDT regime during broker transition
S25	IRS Schedule D instructions	Wash-sale review and recordkeeping requirements
S26	SEC settlement-cycle risk alert	General T+1 settlement framework; account/product availability still verified
S27	TradeZella features	Journal, setup attribution and replay patterns
S28	Balances and positions and transactions	Portfolio source-of-truth endpoints
S29	Instruments and symbology	Broker symbols, metadata and stream identifiers
S30	Market sessions and margin/risk	Session calendars, restrictions and broker margin hypotheses

Evidence versus assumptions

Verified documentation: tastytrade supports the referenced families of reads, order management and streams; certification is not realistic paper execution; the documented backtester is separate; current official OpenAI docs support structured tool/output contracts. Proposed design: our architecture, risk tiers, schedules, sizing policies, data quality rules, UI, backlog, service targets and budgets. To verify at implementation: account-specific approvals, exact request schemas, runtime delivery/order guarantees, historical coverage, vendor rights, exercise/DNE operations and the broker's current day-trading regime.

Important contradictions and limitations retained

Some reference banners mention request use of `ext-client-order-id` while the broker's retry guide identifies it as read-only and points to `external-identifier`. The reference operation index can also differ between older `/open-api-spec/` pages and current `/reference/` pages. Contract tests against the current schema/environment decide enabled capabilities; the application does not silently choose whichever prose looks convenient.

OptionStrat's direct pages returned a fetch restriction; its official indexed feature descriptions supported the limited design-pattern comparison. No claims about proprietary implementation, broker compatibility or purchase pricing are made from that limited access.

Implementation handoff

Build the system in this specification incrementally. The existing deliverables are design assets, starter contracts and a local prototype, not a connected trading engine.

Repository layout

Path	Responsibility
apps/web/src/shell/	Account/environment chrome and navigation
apps/web/src/features/portfolio/	Groups, positions, ownership and inspectors
apps/web/src/features/trade/	Chain, builder, review and lifecycle
apps/web/src/features/research/	Markets, evidence, experiments and replay
apps/web/src/features/risk/	Limits, scenarios, margin and expiry actions
apps/web/src/features/automation /	Strategy recipes, versions and promotions
apps/web/src/features/operations /	Incidents, connection ages and recovery
services/trading/domain/	IDs, commands, value types, invariants and events
services/trading/broker/	OAuth, capabilities, REST/stream adapters and broker gateway
services/trading/market/	Instruments, subscriptions, quality, calendars and snapshots
services/trading/quant/	Pricing, payoff, Greeks, indicators and scenario engines
services/trading/risk/	Policies, authorization, reservations and decisions
services/trading/execution/	Immutable prepared commands, writer lease and reconciliation
services/trading/portfolio/	Ledger, virtual lots, broker truth and performance
services/trading/strategy/	Typed AST, evaluator, sizing and management plans
services/trading/ai/	Context packs, typed read/draft tools, outputs, evals and costs
services/trading/research/	Engine adapters, manifests, experiments and replay
services/trading/communication/	Durable inbox, outbox, channels and escalation
services/trading/api/	Authenticated HTTP/SSE routes and DTO mapping
workers/	Ingestion, strategy, execution, AI, research and notifications endpoints
contracts/	JSON/OpenAPI schemas and generated frontend types
db/migrations/	Production migrations, constraints, grants and append-only policy
tests/fixtures/	Redacted recorded contracts and lifecycle cases
tests/contract/	Certification and provider contract verification
tests/domain/	High-impact numeric, authorization and reservation properties
tests/faults/	Submit boundary, handover, reconnect and restore tests
infra/	Environment templates, observability, secrets and recovery

First vertical slice

Implement a read-only account connection end to end: TokenManager → account discovery → instruments/positions/balances/orders → account stream → normalized snapshots → reconciliation → Portfolio/Overview/Operations UI. Record raw redacted evidence and expose timestamp/reconciliation state. Support a manual broker trade entering Unassigned and a revoked connection refusing new work. No live write credentials in this slice.

Next implement one stock and one ordinary unadjusted option trade analysis: snapshot → exact instruments → payoff/quant → policy checks → local paper intent → simulated order lifecycle → portfolio/journal. Add broker certification preflight/submission only after the typed gateway, reservation and UNKNOWN recovery path are complete.

Implementation instructions

1. Read the entire blueprint and source register before selecting APIs. Validate exact current schemas; documentation conflicts are explicitly retained.
2. Keep server-owned account identity, money, quantity, symbols and authorization authoritative. The UI cannot be the only risk gate.
3. Share a single typed strategy AST and evaluator across replay, paper, shadow and live. Do not implement a visually similar but separate live rules engine.
4. Implement Decimal/tick/quantity conventions before analytics become execution inputs. Every Greek and event field needs units and timestamp semantics.
5. Keep broker submissions single-writer and durable. Outbox/task retries cannot silently repeat a financial POST.
6. Preserve raw broker status, internal intent state and verified filled quantity separately.
7. Never release an ambiguous reservation by timeout alone; do not automatically fail over an account writer until in-flight commands are reconciled.
8. Put the OpenAI assistant behind account-scoped read/draft tools. No broker write tokens, arbitrary URLs or unrestricted shell access in the model interface.
9. Treat absent calendar/data/contract coverage as UNKNOWN. Make unsupported features inspectable and blocked.
10. Match the prototype's information hierarchy and interaction intent, then implement the full screen specification. Do not leave synthetic prices or return charts in a production UI.
11. Distinguish a software exit from a broker-resident protective order. Verify bracket behavior by product and partial-fill state.
12. Use the backlog's dependencies and test matrix to select the next slice. Record source/schema/test evidence for each enabled capability.
13. Ship narrow functional slices. Avoid a placeholder service fleet, dozens of unconnected screens, or untested broad strategy automation.
14. Report supported functionality, validation, limitations and what remains disabled after each release.

Proposed OpenAI call pattern

Use the current official Responses API and supported configurable model. Load `06_trade_proposal.schema.json` as the strict output schema, and provide only curated context/evidence plus allowed read/draft tools. Function tools also use strict schemas. The application handles tool-call dispatch with account permissions, returns trusted results to the model, then validates the final proposal semantically.

Conceptual request fields: `model=<validated configuration>`, `input=<context and question>`, `tools=<strict function definitions>`, `text.format={type: json_schema, name: trade_proposal, strict: true, schema: <loaded schema>}`, and retention configuration appropriate to the selected features. Exact current SDK/request syntax is verified before coding. No model invocation or key setup is required for this design package.

Definition of done for the first live capability

The exact account/environment, product, order type and management path have contract evidence; the policy envelope and reviewed intent are durable; sizing and portfolio checks include existing and pending risk; submit uncertainty and cancel/fill races recover safely; the broker's true state reconciles with the app; independent outage alerts work; restart/restore cannot replay old trades; and the user explicitly activates the concrete limited authority. Strategy edge remains a separate research decision.